

Belegarbeit

zu den Lehrveranstaltungen
Embedded Systems III und *Verteilte Systeme*

Sommersemester 2026

Dozent: Prof. Dr.-Ing. Andreas Pretschner

Benutzeroberfläche zur Steuerung von Roboterarmen

vorgelegt von:	Lennart Mende
Studiengang:	Elektrotechnik und Informationstechnik
Matrikelnummer:	81256
E-Mail-Adresse:	lennart.mende@stud.htwk-leipzig.de

und

vorgelegt von:	Richard Mende
Studiengang:	Elektrotechnik und Informationstechnik
Matrikelnummer:	81169
E-Mail-Adresse:	richard.mende@stud.htwk-leipzig.de

Datum der Abgabe:	29. August 2026
-------------------	-----------------

Inhaltsverzeichnis

Abbildungsverzeichnis	II
Tabellenverzeichnis	II
1 Einführung und Systemübersicht	1
1.1 Systembeschreibung	1
1.2 Zielsetzung	2
1.3 Teilbereiche des Projekts	2
1.4 Gliederung des Abgabeordners	3
2 Grundlagen und Systemarchitektur	4
2.1 SO-ARM101 und Servomotoren	4
2.2 LeRobot-Framework	4
3 Eingebettete Linux-Umgebung	5
3.1 Erstellung eines eigenen BitBake Layers	5
3.2 Erstellung der BitBake Recipes	5
3.2.1 Recipes für Python-Bibliotheken	6
3.2.2 Recipe für Lerobot	9
3.2.3 Recipe für Lerobot-GUI	10
3.3 Erstellen des SD-Karten-Images	11
4 Softwarearchitektur und Steuerung	13
4.1 LeRobot-Modul	13
4.1.1 Teleoperation	14
4.1.2 Trajektorienaufnahme und -wiedergabe	15
4.1.3 Steuerung des Follower-Arms über die Webapp	16
4.2 GUI	16
5 Datenfluss und Kommunikation	22
5.1 Komponenten für die Kommunikation	22
5.2 Abhängigkeiten	23
5.3 Python-seitiger MQTT-Datenfluss	24
5.4 Java-seitiger Datenfluss	26
5.5 REST-API und Spring Boot	27
5.6 Datenformat	28
5.7 MQTT-Broker	29
5.8 Zertifikate	30
6 Web-Dashboard	32
6.1 Überblick	32
6.2 Auto-Refresh der offenen Tabs	34
6.3 Betriebsmodi	34
6.4 Alarm	36
7 Inbetriebnahme	38
7.1 Installation	38
7.2 Einmaliges Dashboard-Setup	38
7.3 Betrieb des gesamten Systems	39

8 Zusammenfassung und Ausblick	40
8.1 Zusammenfassung	40
8.2 Ausblick	40
Literaturverzeichnis	III

Abbildungsverzeichnis

1.1 Aufbau des Robotersystems mit STM	1
4.1 Schematischer Ablauf der Teleoperation	15
4.2 Verschiedene Modi der GUI	16
4.3 Layout der GUI während der Systemeinrichtung	17
4.4 Layout der GUI bei erfolgreicher Systemeinrichtung	18
4.5 Teleoperations-Fenster im Ausgangszustand	19
4.6 Teleoperations-Fenster nach dem Drücken des Start-Knopfs	19
4.7 Record&Replay-Fenster im Ausgangszustand	19
4.8 Record&Replay-Fenster nach dem Drücken des Start-Knopfs	20
4.9 Konfliktfall für die Steuerung der Roboterarme	20
4.10 Steuerung der Roboterarme über die Webanwendung	21
5.1 Systemarchitektur des Datenflusses	22
6.1 Beispielbild für den Reiter General auf der Website	33
6.2 Beispielbild für den Reiter Diagram auf der Website	33
6.3 Beispielbild für den Reiter Control auf der Website	34
6.4 Beispielhafte Alarmmeldung im Reiter General	37

Tabellenverzeichnis

3.1 Installation der Dateien des Recipes für LeRobot auf dem Target-System .	9
3.2 Installation der Dateien des Recipes für die GUI auf dem Target-System .	10
5.1 RESTful-API-Endpoints	28
6.1 Schwellwerte für Alarmmeldungen	36

1 Einführung und Systemübersicht

1.1 Systembeschreibung

Als technische Grundlage dient das Robotersystem SO-ARM101. Der Aufbau des Systems ist in Abbildung 1.1 dargestellt.



Abbildung 1.1: Aufbau des Robotersystems mit STM

Das System besteht aus einem Leader- und einem Follower-Arm mit jeweils fünf Freiheitsgraden sowie einem einseitig aktiven Greifer. Der Leader-Arm ermöglicht eine teleoperative Programmierung des Follower-Arms. Hierzu wird der Leader-Arm manuell geführt, während die Gelenkstellungen quasikontinuierlich erfasst und als Trajektorie aufgezeichnet werden. Der Follower-Arm verwendet diese Daten in einer PID-Regelung als Führungsgrößen, um die aufgezeichneten Bewegungen nachzufahren.

Zur Erweiterung des Robotersystems wurde eine eingebettete Steuerung sowie eine verteilte Architektur zur Überwachung und Visualisierung von Betriebsdaten entwickelt. Die Embedded-Komponente ermöglicht die lokale Bedienung und Steuerung des Roboters über einen STM32MP135F-DK. Die verteilte Architektur dient der Übertragung, Verarbeitung und Visualisierung von Sensordaten sowie der Bereitstellung einer zusätzlichen Steuerungsmöglichkeit über eine Webanwendung.

Das Gesamtsystem kombiniert somit die direkte Steuerung des Roboters über ein eingebettetes System mit einer modularen Softwarearchitektur, bei der Datenerfassung, Verarbeitung und Benutzerinteraktion auf verschiedene Systemkomponenten verteilt werden können.

1.2 Zielsetzung

Das Ziel des Gesamtprojekts ist, das SO-ARM101-Systems um eine lokale Steuerungs- und Bedienmöglichkeit zu erweitern und eine verteilte Architektur zur Überwachung und Visualisierung von Betriebsdaten zu entwickeln.

Im Rahmen des Moduls Embedded Systems III liegt der Fokus auf der Implementierung zweier Steuerungsmethoden für das Robotersystem. Hierzu sollen zwei Betriebsmodi umgesetzt werden: Der erste Modus ermöglicht die direkte teleoperative Programmierung des Follower-Arms durch den Leader-Arm. Der zweite Modus ermöglicht zunächst die Aufnahme einer durch Teleoperation eingeteachten Trajektorie und anschließend die wiederholte Ausführung dieser Bewegung.

Die Auswahl der Steuerungsmodi sowie die Visualisierung von Hinweisen, Statusmeldungen und zusätzlichen Debug-Informationen erfolgt über eine Benutzeroberfläche auf dem integrierten Display des STM32MP135F-DK. Zusätzlich soll die notwendige Laufzeitumgebung bereitgestellt werden, sodass die entwickelten Softwarekomponenten direkt auf dem eingebetteten System ausgeführt werden können.

Im Rahmen des Moduls Verteilte Systeme liegt der Fokus auf der Übertragung, Verarbeitung und Bereitstellung der im Robotersystem gemessenen Daten.

Ziel ist es, die Kommunikation zwischen Roboter, Datenverarbeitung und Benutzerinteraktion durch eine verteilte Architektur zu ermöglichen. Somit können Sensordaten unabhängig von der Robotersteuerung verarbeitet und visualisiert werden. Gleichzeitig wird eine Schnittstelle geschaffen, sodass das Dashboard zur Programmierung der Trajektorie verwendet werden kann.

Durch die Aufteilung in einzelne Systemkomponenten können diese auf ihre jeweiligen Aufgaben spezialisiert sowie unabhängig voneinander erweitert und optimiert werden.

1.3 Teilbereiche des Projekts

Das Gesamtprojekt umfasst zwei fachliche Schwerpunkte aus den Modulen *Embedded Systems III* und *Verteilte Systeme*. Beide Bereiche basieren auf demselben Robotersystem und verwenden teilweise gemeinsame Komponenten. Dennoch unterscheiden sie sich in ihrer Zielsetzung und den verwendeten Technologien. Der Bereich Verteilte Systeme integriert den Teil aus Embedded Systems und baut auf ihm auf.

Der Schwerpunkt des Moduls Embedded Systems III liegt auf der direkten Ansteuerung und Bedienung des Robotersystems mit der Integration des STM32MP135F-DK für Steuerung und Visualisierung. Dazu gehören insbesondere:

- Bereitstellung der Laufzeitumgebung als SD-Karten-Image
- Implementierung der Steuerungsmethoden
- Entwicklung der grafischen Benutzeroberfläche

Der Schwerpunkt des Moduls Verteilte Systeme liegt auf der Umsetzung einer Kommunikations- und Verarbeitungsarchitektur. Hierzu gehören:

- Übertragung von Sensordaten
- Verarbeitung und Bereitstellung der Daten durch ein Backend

- Visualisierung über eine Webanwendung
- Steuerung des Follower-Arms über die Webanwendung

Die Kapitel 3 und 4 sind dem Modul *Embedded Systems III* zuzuordnen, während die Kapitel 5 und 6 die Inhalte des Moduls *Verteilte Systeme* abdecken.

1.4 Gliederung des Abgabeordners

Der Abgabeordner umfasst mehrere Unterordner. In `documentation` liegt die Projekt-Dokumentation im PDF-Format. Der Ordner `meta-lerobot` beinhaltet die benötigten Verzeichnisse und Dateien für die Kapitel 3 und 4 für das Modul *Embedded Systems III*. Die Inhalte für *Verteilte Systeme* sind in den Ordnern `certs`, `mosquitto`, `mqtt` und `springboot` zu finden. Diese sind für die Kapitel 5 und 6 relevant.

Der Abgabeordner dieses Projekts ist außerdem unter dem GitHub-Repository mit dem Link <https://github.com/LennartMende/so101-standalone-ctrl> zu finden. Außerdem wurde ein Demonstrationsvideo sowie das verwendete SD-Karten-Image unter dem Link <https://nextcloud.eit.htwk-leipzig.de/index.php/s/BfE2Ga9As6mmBBd> abgelegt.

2 Grundlagen und Systemarchitektur

2.1 SO-ARM101 und Servomotoren

Die Motoren der Roboterarme sind STS-3215-Servomotoren der Firma FEETECH und verfügen über Sensorik zur Erfassung von Position, Geschwindigkeit, Lastmoment, Eingangsspannung, Stromstärke und Temperatur. Nach [1] weisen die Motoren interne PID-Regler für Position und Geschwindigkeit auf. Einige dieser Größen werden entweder direkt für die Regelung verwendet oder für Analyse und Alarmzwecke genutzt.

2.2 LeRobot-Framework

Das LeRobot-Framework stellt eine API für den High-Level-Zugriff auf die Motoren bereit und ist somit eine Abstraktionsebene. Es stellt Klassen bereit, die den Lebenszyklus des Motorbusses und der Motoren vereinfachen. Dazu zählen die Initialisierung, der Verbindungsaufbau, das Auslesen und Schreiben von Registern sowie der Verbindungsabbau. Es können sowohl einzelne Motoren als auch der gesamte Motorbus angesprochen werden.

Das LeRobot-Modul wird als Git-Repo auf GitHub bereitgestellt. Da in diesem Zusammenhang keine Anwendung von Algorithmen des Machine Learnings auf Basis von Kamerabildern erfolgt, wurde eine ältere LeRobot-Version vom 12.06.2025 als Basis für eigene Entwicklungen verwendet. Dieser Stand enthält alle benötigten Funktionalitäten. Außerdem ist der Umfang deutlich geringer als bei neueren LeRobot Repo-Versionen, wodurch nicht benötigte Funktionalitäten wie die Verarbeitung von Kamerabildern einfacher aus dem Quellcode entfernt werden können. Das Repo wurde mit diesen Befehlen geklont und auf den alten Stand gesetzt:

```
git clone https://github.com/Seeed-Projects/lerobot.git
cd lerobot
git checkout edfebd522c34b9ad4763fb078304b09dce360b04
```

Die vorhandenen Skripte müssen um eigene Methoden ergänzt werden, um Messwerte wie Spannung und Temperatur auslesen zu können. Im Repo sind in den Klassen `S0101_Follower` und `S0101_Leader` jedoch keine Methoden zum Auslesen dieser Messgrößen vorhanden. In den beiden Skripten `lerobot/common/robots/so101_follower/so101_follower.py` und `lerobot/common/teleoperators/so101_leader/so101_leader.py` werden die dafür notwendigen Funktionen implementiert. Zudem werden diese Funktionen als abstrakte Methoden in den zugehörigen Basisklassen `Robots` und `Teleoperators` ergänzt, sodass alle abgeleiteten Klassen ein konsistentes Interface bereitstellen. Die Implementierung orientiert sich an der bestehenden `get_observation`-Methode und nutzt die generische `sync_read`-Funktion der `MotorsBus`-Klasse aus dem LeRobot-Repository. Über diese Schnittstelle lassen sich bei Bedarf auch weitere Größen wie Strom, Geschwindigkeit oder Lastmoment auslesen. Insbesondere Look-Up-Tables und Methoden für das Auslesen der Register für die Messwerte von Position, Temperatur und Spannung sind Bestandteile der API. Nach [2] sind die Registerwerte direkt in die Messwerte der physikalischen Größen umrechenbar:

- Spannung in *Volt*: Registerwert muss durch 10 geteilt werden
- Temperatur in *°C*: Registerwert kann direkt übernommen werden
- Position in *°*: von Registerwert und Kalibrierung abhängig

3 Eingebettete Linux-Umgebung

In diesem Kapitel wird die Erstellung des SD-Karten-Images für den STM32MP135F-DK mithilfe von Yocto und BitBake beschrieben. Ziel ist es, ein reproduzierbares Linux-System zu erzeugen, das alle für den Betrieb der Roboterarme notwendigen Softwarekomponenten enthält.

Die Entwicklung erfolgt in einer Docker-Umgebung, um eine reproduzierbare Build-Umgebung sicherzustellen. Diese Vorgehensweise basiert auf [3]. Das Einrichten der Docker-Umgebung sowie das Klonen des unter [4] bereitgestellten Repository für Mickledore V6.1 werden als Ausgangspunkt vorausgesetzt. Durch das geklonte Repository sind bereits grundlegende Layer vorhanden, die im Weiteren verwendet werden.

3.1 Erstellung eines eigenen BitBake Layers

Zur Integration eigener Softwarekomponenten wird ein separates BitBake-Layer mit dem Namen `meta-lerobot` erstellt. Nachdem der Docker-Container gestartet wurde, kann die Build-Umgebung für BitBake mit

```
source ~/data/yocto-labs/layers/openembedded-core/oe-init-build-env \
build-openstlinuxweston-stm32mp13-disco
```

initialisiert werden. Das benötigte Layer wird mit den Befehlen

```
cd ~/data/yocto-labs/layers
bitbake-layers create-layer --priority 7 meta-lerobot
bitbake-layers add-layer meta-lerobot
```

erstellt und der Build-Umgebung hinzugefügt. Die Priorität kann dabei frei gewählt werden, allerdings hat dieses Layer mit Priorität 7 einen Vorrang gegenüber den Standard-Layern. Dies kann außerdem mit

```
bitbake-layers show-layers
```

überprüft werden, denn dort wird das neu angelegte Layer `meta-lerobot` mit Pfad und Priorität aufgelistet. Diesem erstellten Layer sollen sowohl eigene Anwendungen als auch zusätzliche Python-Bibliotheken hinzugefügt werden. Die Inhalte des Layers können entweder direkt aus dem Abgabeordner im Verzeichnis `meta-lerobot` kopiert oder manuell erstellt werden. Dafür müssen zusätzlich die Zertifikate und Schlüssel nach der Anleitung in Abschnitt 5.8 erstellt und der dabei generierte `certs` Ordner in das Verzeichnis `meta-lerobot/recipes-lerobot/lerobot/files` kopiert werden. Die manuelle Erstellung der Recipes wird in den folgenden Abschnitten beschrieben. Hierbei bauen die beschriebenen Schritte aufeinander auf. Bei einem Neuanfang muss der oben beschriebene Befehl zur Initialisierung der Build-Umgebung für BitBake erneut ausgeführt werden.

3.2 Erstellung der BitBake Recipes

Für dieses Projekt werden die folgenden 3 eigenen Recipes benötigt:

- `recipes-python`: zusätzliche Python-Bibliotheken
- `recipes-lerobot`: Hauptanwendung zur Steuerung der Roboterarme
- `recipes-gui`: grafische Benutzeroberfläche

Darüber hinaus werden bereits existierende Recipes verwendet, die in den vorhandenen Layern enthalten sind. Dazu zählen `python3`, `python3-paho-mqtt`, `python3-pyudev`, `python3-pyserial`, `udev`, `qtwayland`, `qtwayland-plugins`, `weston` und `weston-init`.

Ob die jeweiligen Recipes in den bereits vorhandenen Layern existieren, kann mit

```
bitbake-layers show-recipes <recipe>
```

überprüft werden. Beispielsweise erhält man für

```
bitbake-layers show-recipes python3-pyserial
```

Version 3.5 im Layer `meta-python`.

Die Erklärung für die jeweiligen Recipes ist in den folgenden Unterkapiteln zu finden. Für die Recipes innerhalb dieses Projekts ergibt sich folgender Dependency Tree, wobei hinter den Recipes die jeweils verwendete Versionsnummer angegeben ist:

```
qtwayland-plugins 5.15.9
weston 11.0.1
lerobot-gui
|-- python3 3.11.2
|-- python3-paho-mqtt 1.6.1
|-- python3-pyudev 0.24.0
|-- python3-pyqt5 5.15.9
|-- qtwayland 5.15.9
|-- lerobot
    |-- udev 1:253.1
    |-- weston-init 1.0
    |-- python3-feitech-servo-sdk 1.0.0
        |-- python3-pyserial 3.5
    |-- python3-draccus 0.10.0
        |-- python3-mergedeep 1.3.4
        |-- python3-pyyaml-include 1.4.1
        |-- python3-pyyaml 6.0
        |-- python3-toml 0.10.2
        |-- python3-typing-inspect 0.9.0
            |-- python3-typing_extensions 4.5.0
            |-- python3-mypy_extensions 0.4.3
```

3.2.1 Recipes für Python-Bibliotheken

Für dieses Projekt werden mehrere Python-Bibliotheken benötigt. Während `python3`, `python3-paho-mqtt`, `python3-pyudev`, `python3-pyserial`, `udev`, `weston` und `weston-init`, `qtwayland` und `qtwayland-plugins` bereits in den Layern vorhanden sind, müssen für `python3-feitech-servo-sdk`, `python3-draccus` und `python3-pyqt5` zusätzliche Recipes erstellt werden. Dafür muss zunächst mit

```
cd ~/data/yocto-labs/layers/meta-lerobot
mkdir recipes-python
cd recipes-python
```

der Ordner für die Python-Recipes erstellt und in diesen gewechselt werden. Eine Anleitung zur Erstellung der eigenen Python-Recipes ist in den folgenden Abschnitten zu finden.

Recipe für python3-feetech-servo-sdk

Zunächst werden mit

```
mkdir -p python3-feetech-servo-sdk/files
```

die benötigten Unterordner angelegt. Anschließend kann mit

```
wget -P python3-feetech-servo-sdk/files \
https://files.pythonhosted.org/packages/source/f/feetech-servo-sdk/
feetech-servo-sdk-1.0.0.tar.gz
```

das benötigte Paket für diese Python-Bibliothek heruntergeladen werden. Mithilfe von `recipetool` kann die BitBake Datei für dieses Recipe mit

```
recipetool create -o \
python3-feetech-servo-sdk/python3-feetech-servo-sdk.bb \
python3-feetech-servo-sdk/files/feetech-servo-sdk-1.0.0.tar.gz
```

generiert werden. Dadurch besteht der Vorteil, dass in der BitBake Datei automatisch beispielsweise die Beschreibung oder die Checksum eingetragen werden. Man kann sich den Inhalt der Datei mit

```
vi python3-feetech-servo-sdk/python3-feetech-servo-sdk.bb
```

anzeigen lassen und bearbeiten. Hierbei muss noch die Versionsnummer hinzugefügt werden. Außerdem kann man die Dependencies genauer tracken und stellt fest, dass lediglich `python3-pyserial` benötigt wird, `python3-core` hingegen nicht. Schließlich ergibt sich folgender Inhalt, der ebenfalls aus dem entsprechenden Pfad aus dem Abgabeordner kopiert werden kann:

```
SUMMARY = "This is source code from official feetech repository"
HOMEPAGE = "https://github.com/Adam-Software/FEETECH-Servo-SDK"

LICENSE = "Unlicense"
LIC_FILES_CHKSUM = "file://LICENSE;md5=
d512eeaf0d5285acc53d4569054cbb08"

PV = "1.0.0"

SRC_URI = "file://feetech-servo-sdk-${PV}.tar.gz"

S = "${WORKDIR}/feetech-servo-sdk-${PV}"

inherit setuptools

RDEPENDS:${PN} += "python3-pyserial"
```

Recipe für `python3-draccus`

Für `python3-draccus` werden weitere Python-Bibliotheken benötigt, wie der Dependency Tree aus Kapitel 3.2 zeigt. Hierbei sind die 4 Recipes `python3-typing_extensions`, `python3-mypy_extensions`, `python3-pyyaml` und `python3-toml` bereits in den Layern vorhanden. Die anderen 3 Recipes `python3-mergedeep`, `python3-pyyaml-include` und `python3-typing-inspect` müssen allerdings hinzugefügt werden, sodass schließlich auch das Recipe für `python3-draccus` erstellt werden kann.

Hierbei wird für die zu erstellenden Recipes dieselbe Vorgehensweise verwendet, wie es beim Recipe für `python3-feetech-servo-sdk` bereits beschrieben wurde:

1. Anlegen der Ordnerstruktur mit `recipes-<name>` und dem Unterordner `files`
2. Abspeichern der `*.tar.gz` Datei der Python-Bibliothek in den `files` Ordner
3. Erstellen der BitBake Datei mit den notwendigen Abhängigkeiten

Die Befehle können von oben übernommen werden und müssen entsprechend auf den Namen des Pakets angepasst werden. Die Downloads für die `*.tar.gz` Dateien der 4 genannten Bibliotheken sind folgende:

```
python3-draccus:
https://files.pythonhosted.org/packages/source/d/draccus/
draccus-0.10.0.tar.gz

python3-mergedeep:
https://files.pythonhosted.org/packages/source/m/mergedeep/
mergedeep-1.3.4.tar.gz

python3-pyyaml-include:
https://files.pythonhosted.org/packages/source/p/pyyaml-include/
pyyaml-include-1.4.1.tar.gz

python3-typing-inspect:
https://files.pythonhosted.org/packages/source/t/typing-inspect/
typing-inspect-0.9.0.tar.gz
```

Die einzelnen BitBake Dateien für die 4 Recipes sind im Abgabeordner zu finden, sodass auch das Recipe für `python3-draccus` mit allen Abhängigkeiten zusammengestellt werden kann.

Recipe für `python3-pyqt5`

Für die GUI wird zusätzlich das Recipe `python3-pyqt5` benötigt. Dieses Recipe verfolgt nicht dieselbe Vorgehensweise wie die anderen Recipes für die Python-Pakete. Hierfür werden die 3 in [5] gegebenen BitBake Dateien `python3-pyqt-builder-native_1.15.0.bb`, `python3-pyqt5-sip_12.12.1.bb` und `python3-pyqt5_5.15.9.bb` in den zuvor zu erstellenden Recipe-Ordner `python3-pyqt5` kopiert.

3.2.2 Recipe für Lerobot

Neben den benötigten Python-Bibliotheken wird auch das **lerobot**-Python-Paket in das erzeugte Linux-System integriert. Das Verzeichnis enthält die folgenden Komponenten:

- Skripte für die Durchführung der Teleoperation und des Record & Replay im Verzeichnis **lerobot/lerobot**
- für die Kalibrierung benötigte Skripte im Verzeichnis **lerobot/calibration**
- für die Kommunikation mit den Roboterarmen benötigte Zertifikate im Verzeichnis **certs**
- Regeln für die USB-Ports im Verzeichnis **scripts**

Zunächst wird ins Layer **meta-lerobot** gewechselt und die benötigten Verzeichnisse angelegt. Dafür werden die Befehle

```
cd ~/data/yocto-labs/layers/meta-lerobot
mkdir -p recipes-lerobot/lerobot/files
```

ausgeführt. In das **files** Verzeichnis werden anschließend die Inhalte des Verzeichnisses **meta-lerobot/recipes-lerobot/lerobot/files/lerobot** aus dem Abgabeordner kopiert. Hierbei muss der Ordner **certs** mit den oben erstellten Zertifikaten hinzugefügt werden. Außerdem ist eine BitBake Datei erforderlich, welche dafür sorgen soll, dass die Dateien an den richtigen Speicherorten auf dem Target landen. Diese wird mit

```
vi recipes-lerobot/lerobot/lerobot.bb
```

erstellt. Anschließend wird der Inhalt dieser Datei auf den Inhalt der BitBake-Datei **meta-lerobot/recipes-lerobot/lerobot/lerobot.bb** aus dem Abgabeordner erweitert. In dieser BitBake-Datei wird definiert, welche Inhalte aus dem **files** Ordner in welches Zielverzeichnis auf dem STM installiert werden. Diese Übersicht ist in Tabelle 3.1 zu finden. Der dabei angegebene Pfad des Abgabeordners in der 2. Spalte ist jeweils der relative Pfad zu **meta-lerobot/recipes-lerobot/lerobot/files**.

Tabelle 3.1: Installation der Dateien des Recipes für LeRobot auf dem Target-System

Beschreibung	Verzeichnis im Abgabeordner	Zielverzeichnis auf dem Target
lerobot-Python-Paket	lerobot/lerobot	/usr/lib/python3.11/site-packages/lerobot
Kalibrierungsdateien	lerobot/calibration	/usr/share/lerobot/calibration
Zertifikate	certs	/usr/lib/python3.11/site-packages/certs
Regeln für USB-Ports	scripts	/etc/udev/rules.d

In Tabelle 3.1 ist ersichtlich, dass die Inhalte dieses Recipes in unterschiedliche Zielverzeichnisse auf dem STM als Target-System installiert werden. Das **lerobot**-Python-Paket liegt bereits als Quellcode vor. Damit beispielsweise Skripte wie **teleoperate.py** direkt als Python-Modul ausgeführt werden können, werden sie auf dem STM als Python-Paket

installiert. Dies gilt auch für die Zertifikate. Das Skript `99-lerobot.rules` im Zielverzeichnis `/etc/udev/rules.d` sorgt dafür, dass die über USB angeschlossenen Geräte anhand ihrer Seriennummer identifiziert werden können. Dadurch sind Follower und Leader über die USB-Ports `/dev/so-leader` und `/dev/so-follower` ansprechbar. Die Kalibrierungsskripte dienen dazu, die Ruhepose des Followers festzulegen und die beiden Roboterarme korrekt zu kalibrieren.

Abschließend müssen die Zertifikate und Schlüssel mit den korrekten Nutzer- und Gruppenrechten versehen werden. Hierfür wird in der BitBake-Datei zusätzlich `weston-init` als `RDEPENDS` hinzugefügt. Im Recipe wird anschließend die Gruppe `weston` angelegt und als Gruppe für die Zertifikate und Schlüssel verwendet. Darüber hinaus werden die weiteren Laufzeitabhängigkeiten `python3-draccus`, `python3-feetech-servo-sdk` und `udev` als `RDEPENDS` ergänzt.

3.2.3 Recipe für Lerobot-GUI

Für die grafische Benutzeroberfläche zur Steuerung der Roboterarme wird ein weiteres Recipe benötigt. Dieses enthält die folgenden Komponenten:

- Python-Skripte für die GUI im Verzeichnis `src`
- auf der GUI verwendete Bilder im Verzeichnis `images`
- Skripte zum Setzen der Systemzeit und automatischem Starten der GUI im Verzeichnis `scripts`

Anschließend wird nach derselben Vorgehensweise wie beim Recipe für das `lerobot-Python-Package` fortgefahren. Dafür werden mit

```
cd ~/data/yocto-labs/layers/meta-lerobot
mkdir -p recipes-gui/lerobot-gui/files
```

die benötigten Verzeichnisse erstellt und anschließend in die Datei

```
vi recipes-gui/lerobot-gui/lerobot-gui.bb
```

der Inhalt der BitBake-Datei `meta-lerobot/recipes-gui/lerobot-gui/lerobot-gui.bb` aus dem Abgabeordner kopiert. In dieser BitBake-Datei wird definiert, welche Inhalte aus dem `files` Ordner in welches Zielverzeichnis auf dem STM installiert werden. Diese Übersicht ist in Tabelle 3.2 zu finden. Der dabei angegebene Pfad des Abgabeordners in der 2. Spalte ist jeweils der relative Pfad zu `meta-lerobot/recipes-gui/lerobot-gui/files`.

Tabelle 3.2: Installation der Dateien des Recipes für die GUI auf dem Target-System

Beschreibung	Verzeichnis im Abgabeordner	Zielverzeichnis auf dem Target
Python-Skripte für die GUI	<code>src</code>	<code>/opt/lerobot-gui/src</code>
Bilder für die GUI	<code>images</code>	<code>/opt/lerobot-gui/images</code>
Service-Dateien	<code>scripts</code>	<code>/lib/systemd/system</code>

Die Service-Datei `lerobot-gui.service` im Zielordner `/lib/systemd/system` startet die GUI automatisch direkt nach dem Startvorgang des STM. Auch nach einem Absturz der

GUI wird diese automatisch wieder neu gestartet. Die GUI wird vom Benutzer `weston` innerhalb der `wayland` Sitzung ausgeführt. Das Skript `set-system-time.service` im selben Zielordner setzt beim Systemstart die Systemzeit auf den 01.08.2026 00:00:00 Uhr. Dies ist erforderlich, weil die Zertifikate nur innerhalb eines definierten Zeitraums gültig sind und die Systemzeit daher innerhalb dieses Zeitraums liegen muss. Bei bestehender Netzwerkverbindung ist das Setzen der Systemzeit nicht erforderlich, da der STM die Systemzeit automatisch synchronisiert. Das Skript stellt darüber hinaus aber auch ohne bestehende Internetverbindung eine für die Zertifikatsprüfung gültige Systemzeit sicher.

Außerdem werden in der BitBake-Datei `python3`, `python3-paho-mqtt`, `python3-pyudev`, `python3-pyqt5`, `qtwayland` und `lerobot` als `RDEPENDS` hinzugefügt.

3.3 Erstellen des SD-Karten-Images

Nachdem alle benötigten Recipes erstellt und dem Layer `meta-lerobot` hinzugefügt wurden, erfolgt die Erstellung des vollständigen SD-Karten-Images. Dafür wird mit

```
cd ~/data/yocto-labs/build-openstlinuxweston-stm32mp13-disco
```

in die Build-Umgebung gewechselt. Anschließend kann man mit

```
vi conf/local.conf
```

die Konfigurationsdatei öffnen. Am Ende dieser Datei werden die Zeilen

```
IMAGE_INSTALL:append = " lerobot-gui qtwayland-plugins weston"
IMAGE_ROOTFS_MAXSIZE = "1000000"
EXTRA_OEMESON:append:pn-weston = " -Dtools=debug"
```

ergänzt. In der ersten Zeile werden die zusätzlich in das Image aufzunehmenden Pakete angegeben. Dabei zieht `lerobot-gui` die weiteren benötigten Abhängigkeiten über die in der eigenen BitBake-Datei definierten `RDEPENDS` nach. Durch die zweite Zeile wird die maximal zulässige Größe des Root-Dateisystems erhöht. Mit der letzten Zeile werden die benötigten Debug-Werkzeuge aktiviert. Dies ist erforderlich, falls wie für dieses Projekt Screenshots auf dem STM angefertigt werden sollen. Nachdem diese Änderungen an den Recipes und der `local.conf` vorgenommen wurden, kann das Image mit

```
bitbake st-image-weston
```

erstellt werden. Nach Abschluss des Build-Vorgangs kann anschließend überprüft werden, ob die benötigten Dateien erzeugt wurden. Dafür wird zunächst mit

```
cd tmp-glibc/deploy/images/stm32mp13-disco
```

das Verzeichnis gewechselt. Dort kann mit

```
ls -lh *.ext4
```

überprüft werden, ob die 4 benötigten `*.ext4` Dateien für das Boot-FS, User-FS, Vendor-FS und Root-FS erstellt wurden. Die erzeugten `*.ext4`-Dateien können nicht direkt als vollständiges SD-Karten-Image verwendet werden. Stattdessen wird eine `.raw`-Datei benötigt, welche neben dem Root-Dateisystem auch die für den STM benötigten Partitionen und Boot-Komponenten enthält. Die dafür benötigte Flashlayout-Datei kann mit

```
cd flashlayout_st-image-weston/extensible
ls -lh
```

gesucht werden. Wenn die Datei `FlashLayout_sdcard_stm32mp135f-dk-extensible.tsv` in diesem Verzeichnis liegt, kann die für das SD-Karten-Image benötigte `*.raw` Datei mit den Befehlen

```
cd ../../scripts
./create_sdcard_from_flashlayout.sh \
../flashlayout_st-image-weston/extensible/FlashLayout*.tsv
```

erstellt werden. Wenn dieser Schritt erfolgreich war, dann sollte nach

```
cd ..
ls -lh *.raw
```

als Ergebnis die Datei `FlashLayout_sdcard_stm32mp135f-dk-extensible.raw` mit aktuellem Datum erscheinen. Für das Flashen dieser Datei auf die SD-Karte wird die Docker-Umgebung mit `exit` geschlossen und auf dem Host-System fortgefahren. Dafür wird zunächst in das Verzeichnis gewechselt, in dem die `*.raw` Datei liegt. Man kann beispielsweise mit

```
cd ~/projects/STM32/yocto/shared-container-data/yocto-labs/build-open
stlinuxweston-stm32mp13-disco/tmp-glibc/deploy/images/stm32mp13-disco
ls -lh *.raw
```

überprüfen, ob diese Datei auch wirklich existiert. Anschließend muss ein SD-Kartenleser mit der SD-Karte, auf die das Image geschrieben werden soll, mit dem Host-System verbunden werden. Ist dies erfolgreich, dann sollten beim Befehl

```
lsblk
```

unter `sdb` die Partitionen der angeschlossenen SD-Karte angezeigt werden. Anschließend müssen diese Partitionen mit

```
sudo umount /dev/sdb*
```

ausgehängt werden, sodass schließlich die SD-Karte mit

```
sudo dd if=FlashLayout_sdcard_stm32mp135f-dk-extensible.raw \
of=/dev/sdb bs=8M conv=fdatsync status=progress
```

geflasht werden kann. Nach Abschluss des Schreibvorgangs wird mit

```
sync
```

zunächst sichergestellt, dass alle noch ausstehenden Schreibvorgänge abgeschlossen sind. Anschließend kann die SD-Karte mit

```
sudo eject /dev/sdb
```

sicher ausgeworfen werden. Die SD-Karte kann nun in den STM32MP135F-DK eingesetzt werden. Nach Abschluss des Bootvorgangs startet die GUI automatisch.

4 Softwarearchitektur und Steuerung

Dieses Kapitel beinhaltet die Beschreibung der Softwarekomponenten dieses Projekts. Dafür werden das LeRobot-Modul im Zielverzeichnis `/usr/lib/python3.11/site-packages/lerobot` und die Dateien für die GUI im Zielverzeichnis `/opt/lerobot-gui` unterschieden. Die Skripte innerhalb dieser Verzeichnisse werden in den folgenden Abschnitten erklärt.

4.1 LeRobot-Modul

Das LeRobot-Modul im Zielverzeichnis `/usr/lib/python3.11/site-packages/lerobot` beinhaltet die folgenden Verzeichnisse und Skripte:

```
common
|-- motors
|-- robots
|-- teleoperators
|-- utils
|-- constants.py
|-- errors.py
configs
|-- control.py
|-- parser.py
|-- types.py
__init__.py
__version__.py
constants.py
follower_pos_publisher.py
motor_read_write.py
record_and_replay.py
teleoperate.py
teleoperate_with_sliders.py
utils.py
```

Hierbei werden Skripte, die ausschließlich zum Debugging dienen, nicht aufgelistet.

Der `common`-Ordner wurde dem Repo entnommen und die nicht benötigten Dateien entfernt. Dieser enthält mit `constants.py` ein Skript zur Definition von Konstanten sowie mit `errors.py` ein Skript zur Definition von Fehlern. Außerdem werden weitere Unterordner mit den folgenden Inhalten bereitgestellt:

- `motors`: Klassen, Routinen und Konstanten zur Abstraktion der Ansteuerung der FEETECH-Motoren
- `robots`: Klassen und Routinen zur Regelung, Winkelmessung, zum Verbindungsaufbau und weiteren Funktionen des Follower-Arms, darüber hinaus wurden eigene Methoden zur Messung von Temperatur und Spannung hinzugefügt
- `teleoperators`: Klassen und Routinen für Leader-Arm analog zu Follower-Arm in `robots`
- `utils`: Hilfsklassen und -routinen

Der `configs`-Ordner enthält die drei Konfigurationsdateien `control.py`, `parser.py` und `types.py` und wurde ebenfalls aus dem Repo übernommen.

Das Skript `__version__.py` ist zur Untersuchung der LeRobot-Version erforderlich und `__init__.py` dient zur Initialisierung des Moduls.

Die weiteren Skripte dieses `lerobot`-Ordners sind für die Steuerung der Roboterarme verantwortlich. Das Skript `motor_read_write.py` stellt Funktionen bereit, um den Lesezugriff auf beide Arme sowie den Schreibzugriff auf den Follower-Arm zu testen. Wenn das Skript direkt aufgerufen wird, dann werden erst die Gelenke beider Arme gelesen und anschließend wird der Follower in die Ruhepose gesetzt. Dies ist sowohl beim initialen Setup als auch zum Resetten zu Beginn der Steuerungsmethoden erforderlich.

Die Skripte `record_and_replay.py`, `teleoperate.py` und `teleoperate_with_sliders.py` implementieren die 3 Steuerungsmethoden. Diese Skripte werden als Python-Modul importiert. Somit wird auf die Hardware-Abstraktion der LeRobot-Python-Bibliothek zurückgegriffen, wodurch Hardware-Zugriffe vereinfacht werden (siehe Abschnitt 2.2). Die Erklärung dieser Skripte erfolgt in den folgenden Abschnitten.

4.1.1 Teleoperation

Bei der Teleoperation steuert der Bediener den Leader-Arm manuell und der Follower folgt der Trajektorie des Leader-Arms. Die dafür notwendige Logik enthält das Skript `teleoperate.py`. Als Grundlage zur Entwicklung diente das Skript `teleoperate.py` aus dem LeRobot-Repo, das Skript musste jedoch an die Anforderungen angepasst und erweitert werden.

Zu Beginn wird versucht, die Verbindung zu den MQTT-Clients für die Übertragung der physikalischen Größen mit dem MQTT-Broker herzustellen. Zudem wird der Motorbus für beide Arme initialisiert.

Um die Teleoperation durchführen zu können, wird die Bahn des Leader-Arms mit dem Befehl `action = teleop.get_action()` quasikontinuierlich aufgenommen und abgespeichert. Falls sich die MQTT-Clients anfangs mit dem Broker verbinden konnten, werden ebenfalls Spannung und Temperatur der Motoren der beiden Arme aufgenommen und zusammen mit den Positionen über MQTT veröffentlicht. Die aufgenommene Aktion wird mit `robot.send_action(action)` an den Follower-Arm gesendet und dort als Führungsgröße gesetzt. Dadurch kann die PID-Regelung für jedes separate Gelenk erfolgen. Falls die Loop-Zeit zu gering ist, um die vorgegebene Regel-Frequenz einzuhalten, wird bis zum Erreichen der vorgegebenen Loop-Zeit gewartet. Dieser Loop wird so lange wiederholt, bis er explizit abgebrochen wird, da für die Teleoperation keine maximale Ausführungsdauer vorgegeben ist.

Dieser Ablauf ist in Abbildung 4.1 abgebildet: Die roten Übergänge kennzeichnen Fehlerpfade, die durch prozessterminierende Signale (z. B. `KeyboardInterrupt`) oder Kommunikationsfehler (z. B. nicht erreichbarer MQTT-Broker) ausgelöst werden. Schwarze Pfeile repräsentieren reguläre sequentielle Übergänge, die nach Abschluss des vorherigen Schritts erfolgen. Der Rücksprung „wenn Zyklusdauer überschritten“ stellt eine zyklische Wiederholung dar, die nach Ablauf der definierten Zyklusdauer erfolgt.

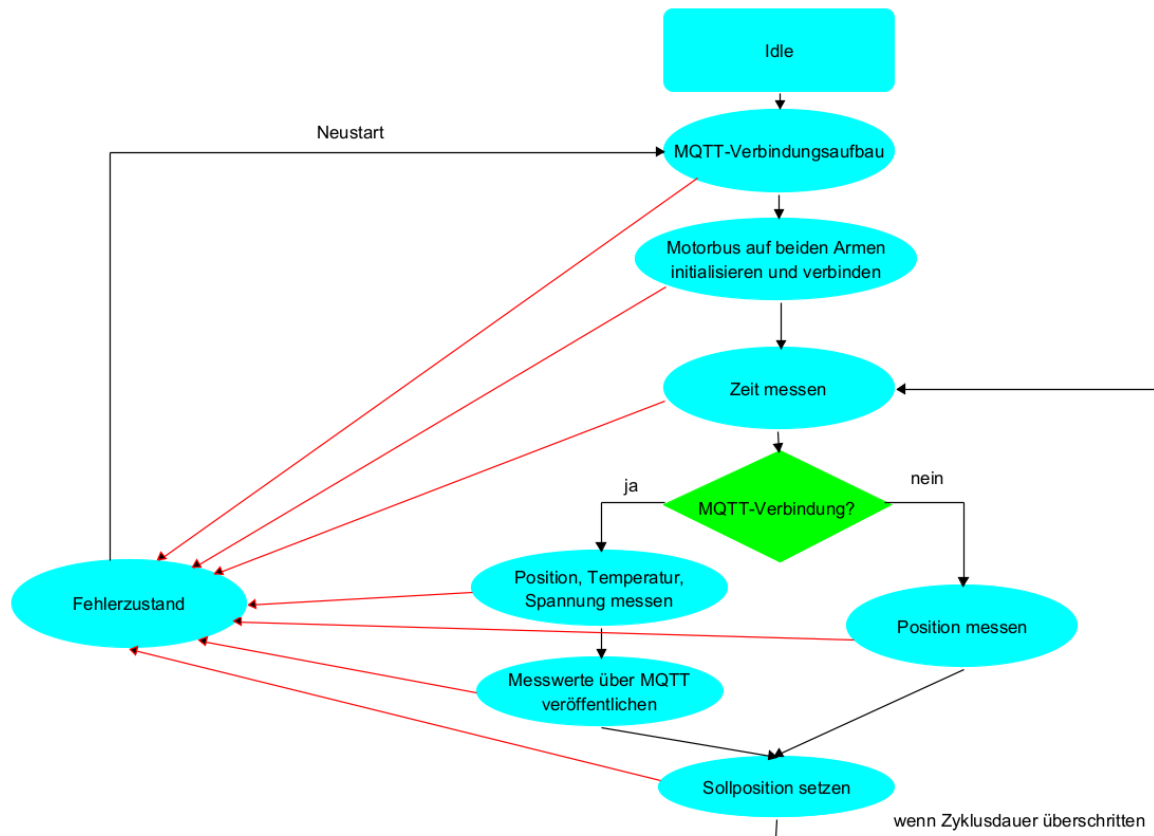


Abbildung 4.1: Schematischer Ablauf der Teleoperation

4.1.2 Trajektorienaufnahme und -wiedergabe

Das Ziel der im Skript `record_and_replay.py` implementierten Steuerungsmethode ist, eine Trajektorie aufzunehmen und diese anschließend möglichst exakt erneut abzuspielen. Als Grundlage für den Aufnahmemodus dient das Skript `teleoperate.py` aus dem LeRobot-Repo. Das Abspeichern und Wiederholen der Trajektorie wurde selbstständig implementiert.

Nach dem Setzen der Ruhepose wird der Roboter im *Record*-Teil (Aufnahme der Trajektorie) teleoperativ gesteuert. Hierfür wird das in Abschnitt 4.1.1 beschriebene Skript ausgeführt. Allerdings wird dabei eine Episodendauer spezifiziert, welche die Dauer der Teleoperation festlegt. Währenddessen werden die Positionswerte des Leader-Arms als Schlüssel-Wert-Paare gespeichert und in einer Liste abgelegt.

Anschließend wird der Arm wieder in die Ruheposition gesetzt, damit der sich folgende *Replay*-Teil (Wiederholung der Trajektorie) in der gleichen Ausgangspose beginnt wie der *Record*-Teil. Im *Replay*-Teil wird dieselbe Struktur wie im *Record*-Teil verwendet, allerdings werden dabei die Leader-Positionen durch die aufgenommene Trajektorie ersetzt. Um die Geschwindigkeit in beiden Teilen annähernd konstant zu halten, entspricht die Loop-Zeit im *Replay*-Teil der durchschnittlichen Loop-Zeit des *Record*-Teils. Andernfalls wäre die Loopdauer beim Abspielen kürzer als beim Aufnehmen, da keine Messwertübertragung über MQTT stattfindet.

4.1.3 Steuerung des Follower-Arms über die Webapp

Im Skript `teleoperate_with_sliders.py` wird ein Subscriber auf dem Topic `control` erstellt. Dieser empfängt die Control-Daten, die mit den Slidern in der Webapp eingestellt und über MQTT Java-seitig veröffentlicht werden, siehe Abschnitt 5.4. Diese Werte werden als Führungsgröße auf die Motoren geschrieben, sodass der Follower-Arm über die Webapp gesteuert werden kann.

4.2 GUI

Das Zielverzeichnis `/opt/lerobot-gui` beinhaltet die folgenden Verzeichnisse und Skripte:

```
images
|-- logo_HTWK.png
|-- setup_button_black.png
|-- setup_button_orange.png
|-- start_button.png
|-- stop_button.png
src
|-- app.py
|-- concurrent_dashboard_subscriber.py
|-- constants.py
|-- control_conflict_window.py
|-- dashboard_mode_control.py
|-- dashboard_subscriber.py
|-- main_window.py
|-- publisher.py
|-- record_replay_window.py
|-- setup_window.py
|-- teleop_window.py
```

Hierbei werden Skripte, die ausschließlich zum Debugging dienen, nicht aufgelistet. Alle weiteren Skripte wurden eigenständig erstellt.

Diese GUI-Skripte dienen zur Bedienung der Roboterarme über das Display des STM. Im Allgemeinen gibt es 3 verschiedene Tabs, zwischen denen der Anwender jederzeit wechseln kann. Die Zusammenwirkung der 3 Modi Teleoperation, Record & Replay und Setup in den jeweiligen Tabs ist in Abbildung 4.2 dargestellt.

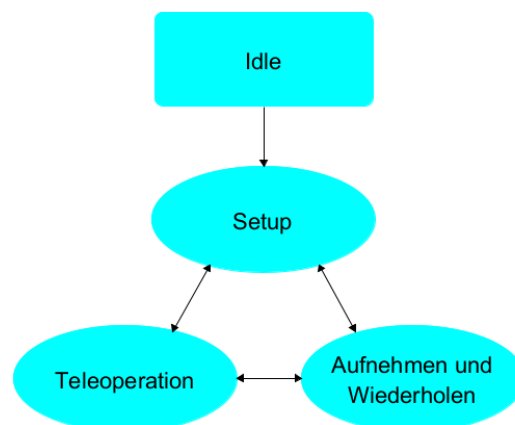


Abbildung 4.2: Verschiedene Modi der GUI

Die für die GUI relevanten Konstanten sind im Skript `constants.py` hinterlegt.

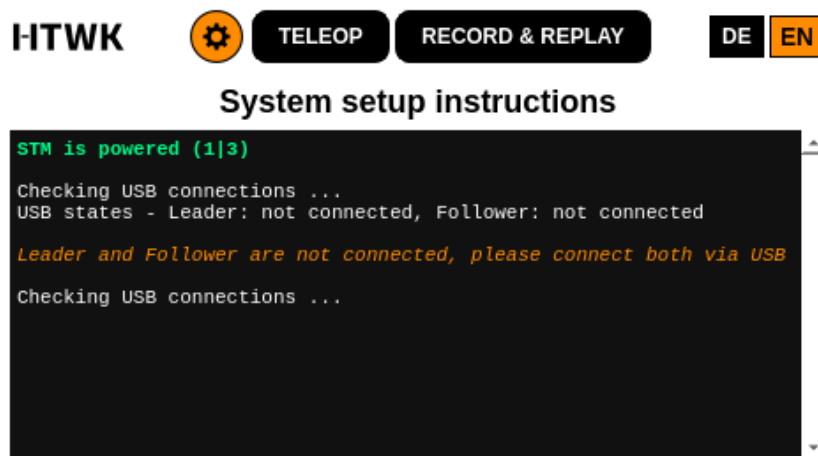


Abbildung 4.3: Layout der GUI während der Systemeinrichtung

Abbildung 4.3 zeigt das Layout der GUI im Setup-Modus und beinhaltet ebenfalls das einheitliche Layout der GUI für alle 3 Tabs. Das Layout der GUI und die allgemeine Steuerung sind im Skript `main_window.py` definiert. Das HTWK Logo in der linken oberen Ecke und das Zahnrad als Symbol für die Systemeinrichtung werden aus dem `images` Ordner geladen. Der aktive Tab ist immer mit orangenem Hintergrund und schwarzer Schrift sowie Umrandung gekennzeichnet. In diesem Fall sind Teleop und Record & Replay die inaktiven Tabs und deshalb mit weißer Schrift auf schwarzem Hintergrund dargestellt. In der rechten oberen Ecke kann die Sprache zwischen Deutsch und Englisch gewechselt werden. Hier gilt dasselbe Farbmuster wie für die Tabs. Wird durch den Bediener der Tab gewechselt, wird der laufende Prozess automatisch beendet, damit die Kontrolle beim neu geöffneten Fenster liegt.

Das Skript `setup_window.py` generiert ein Command-Line-ähnliches Fenster im HTML-Style für die Systemeinrichtung. Das in Abbildung 4.3 abgebildete Fenster wird nach dem Startprozess auf dem STM angezeigt. Der Bediener wird durch die Systemeinrichtung geführt und es werden Hinweise gegeben, welche Schritte durchgeführt werden müssen, um alle Voraussetzungen für die Steuerung der Roboterarme zu erfüllen. Dabei werden die folgenden 3 Kriterien überprüft:

1. Spannungsversorgung des STM
2. USB-Verbindung zwischen Roboterarmen und STM32
3. Spannungsversorgung der Arme, Lesezugriff auf den Leader-Arm sowie Lese- und Schreibzugriff auf den Follower-Arm

Die Spannungsversorgung des STM ist stets gegeben, wenn die GUI angezeigt wird. Für die Überprüfung der USB-Verbindungen wird im Skript `publisher.py` eine eigene Klasse `USBPublisher` als `QThread` verwendet. Mithilfe von `pyudev` werden Änderungen an seriellen Geräten überwacht. Zusätzlich erfolgt unabhängig von solchen Ereignissen eine regelmäßige Überprüfung im Abstand von etwa einer Sekunde. Dadurch kann bestimmt werden, ob Leader- und Follower-Arm über die Ports `/dev/so-leader` und `/dev/so-follower` verbunden sind. Wird einer der beiden Arme über USB verbunden oder getrennt, wird

dies übermittelt und auf der GUI angezeigt. Erst wenn beide Arme über USB verbunden sind, wird dieser Schritt als erfolgreich abgeschlossen angezeigt.

Anschließend erfolgt die Überprüfung der Stromversorgung sowie der Lese- und Schreibzugriffe über den `ServoPublisher` als `QThread` aus dem Skript `publisher.py`. Dafür wird einmal pro Sekunde die Funktion `check_connection()` aufgerufen. Diese versucht, eine Verbindung zu beiden Roboterarmen aufzubauen. Ist dies erfolgreich, werden beide Verbindungen als funktionsfähig erkannt. Tritt dabei ein Fehler auf, werden die Verbindungen als nicht verfügbar betrachtet. Wenn beide Arme verbunden werden können, wird anschließend versucht, die Werte der Servomotoren von Leader und Follower zu lesen und den Follower in die Ruhepose zu fahren. Falls alle Schritte erfolgreich sind, ist die Systemeinstellung vollständig abgeschlossen und das Setup-Window zeigt die Ausgaben aus Abbildung 4.4. Andernfalls werden Fehlermeldungen oder Hinweise für den Bediener ausgegeben.

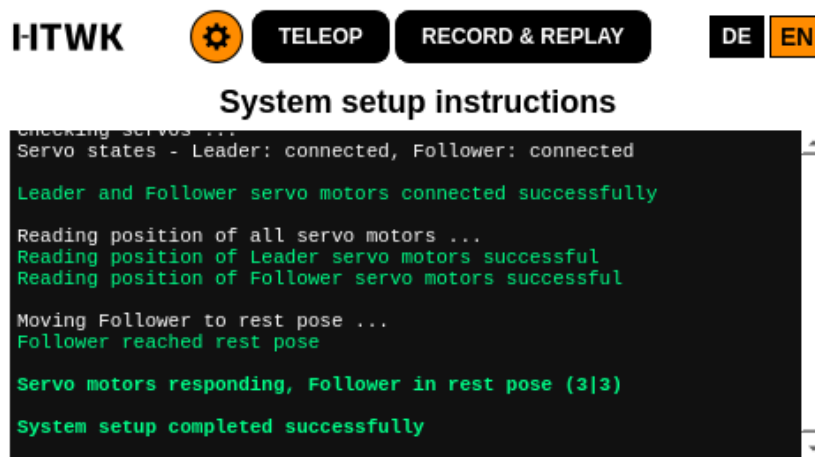


Abbildung 4.4: Layout der GUI bei erfolgreicher Systemeinstellung

Das Skript `teleop_window.py` erstellt ein vertikales Box-Layout. Alle Objekte, die dort hinzugefügt werden, werden übereinander angeordnet. Anschließend werden zwei `QLabel`, ein `QPushButton` und ein weiteres `QLabel` hinzugefügt. Die Label werden zur Anzeige der letzten durchgeführten Aktion, des Events, das beim Drücken des Knopfs ausgelöst wird, und des aktuellen Status verwendet. Über der Statusanzeige wird außerdem ein `QPushButton` hinzugefügt. Beim Öffnen des Teleoperationsfensters erscheint zunächst ein Startknopf. Wird dieser gedrückt, startet ein `QProcess` das in Abschnitt 4.1.1 beschriebene Skript. Die Teleoperation kann durch das Betätigen des Stopp-Knopfs beendet werden. Der `QProcess` wird mit der `terminate`-Methode beendet und der Knopf zeigt wieder ein Start-Symbol an. Außerdem ändern sich die Inhalte der `QLabel`. Abbildung 4.5 zeigt das Teleoperations-Fenster zum Zeitpunkt, wenn erstmals in dieses Fenster gewechselt wird. Abbildung 4.6 veranschaulicht das Fenster für den Zustand nach dem Drücken des Startknopfs. Dabei wurde zusätzlich die Sprache von Englisch auf Deutsch umgestellt.

Für das Skript `record_replay_window.py` gelten dieselben Beschreibungen für das GUI-Layout wie für die Teleoperation. Hier wird jedoch das Skript `record_and_replay.py` in einem `QProcess` gestartet. Die Abbildungen 4.7 und 4.8 zeigen das Layout im Ausgangszustand bzw. nach dem Drücken des Startknopfs.



Abbildung 4.5: Teleoperations-Fenster im Ausgangszustand



Abbildung 4.6: Teleoperations-Fenster nach dem Drücken des Start-Knopfs



Abbildung 4.7: Record&Replay-Fenster im Ausgangszustand

Recording started

Press the button to abort recording

***Recording continues for 19 s***

Abbildung 4.8: Record&Replay-Fenster nach dem Drücken des Start-Knopfs

Das Skript `dashboard_mode_control.py` steuert den Zugriff auf die Roboterarme. Zuerst wird versucht, einen MQTT-Client auf dem Topic `dashboard/mode` mit dem Broker zu verbinden. Wenn der Verbindungsversuch fehlschlägt, wird das Skript `/opt/lerobot-gui/src/app.py` gestartet. In diesem Fall hat der STM die vollständige Kontrolle über die Bewegung der Roboterarme. Falls die Verbindung des MQTT-Clients erfolgreich ist, bestimmt der Dashboard-Modus (siehe Abschnitt 6.3), was auf dem Display des STMs angezeigt wird und welche Komponente die Roboterarme steuert. Entsprechend des Dashboard-Modus wird entweder `/opt/lerobot-gui/src/app.py` oder `/opt/lerobot-gui/src/control_conflict_window.py` ausgeführt.

Das Skript `app.py` instanziert ein `MainWindow`-Objekt. Wenn es interpretiert wird, startet die GUI im Setup-Modus und kann wie oben beschrieben bedient werden. Das Skript `control_conflict_window.py` erzeugt ein weißes Fenster mit dem Hinweis, warum der Konflikt auftritt. Dies zeigt Abbildung 4.9. Wird der Follower-Arm über die Webanwendung im Reiter **Control** gesteuert (siehe Abschnitt 4.1.3), erscheint auf der GUI der Hinweis in Abbildung 4.10.

```

There is a conflict caused by the opened
    browser windows.
    You have to follow those rules:
1. You can't have a Control tab open at
    the same time as an Index or Diagram
        tab.
2. You can't have multiple control
    windows opened at the same time.
  
```

Abbildung 4.9: Konfliktfall für die Steuerung der Roboterarme

You can control the follower arm by
moving the sliders.

Abbildung 4.10: Steuerung der Roboterarme über die Webanwendung

5 Datenfluss und Kommunikation

5.1 Komponenten für die Kommunikation

Es wurde ein Robotik-Telemetrie-System implementiert. Dieses besteht aus den folgenden Komponenten:

- Python-MQTT-Publisher für Positions-, Temperatur- und Spannungsdaten sowie den System-Zustand
- Java-Subscriber für diese veröffentlichten Daten sowie REST-API-Endpoints
- Python-MQTT-Subscriber und Java-MQTT-Publisher für den Dashboard-Zustand
- Java Spring Boot-Backend (MQTT-Subscriber, REST-API, Datenpuffer)
- (zusätzlich Python-Subscriber- und -Publisher zum Debuggen)
- Web-Dashboard (HTML/CSS/JS + Chart.js für Live-Tracking) mit Alarmierung beim Über- und Unterschreiten von Grenzwerten

Die grundlegende Systemarchitektur ist in Abbildung 5.1 dargestellt. Auf der rechten Seite ist das System mit der eingebetteten Steuerung dargestellt, wobei der STM32 über die Python-MQTT-Clients verfügt. Der Broker ist der Mosquitto-Server. Die linke Seite stellt die Verarbeitung auf dem PC mit Dashboard, REST-API, Spring-Controllern und Java-Backend dar.

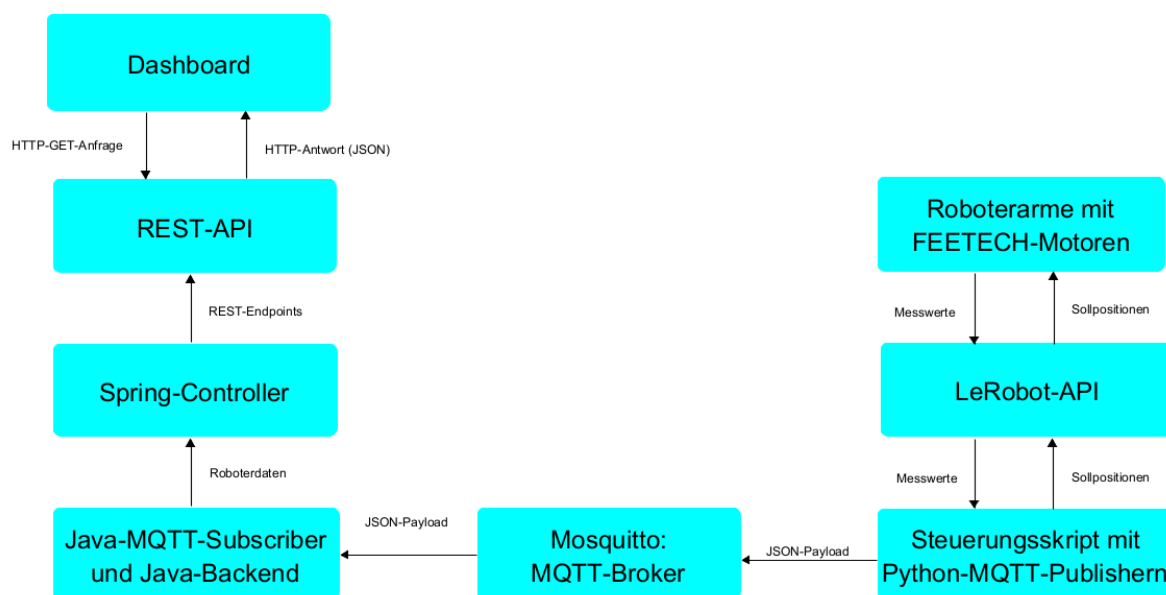


Abbildung 5.1: Systemarchitektur des Datenflusses

5.2 Abhängigkeiten

Es sind diverse Abhängigkeiten erforderlich, die sich auf mehrere Ebenen verteilen:

- Python: Paho-MQTT-Client (beinhaltet Bibliothek), installierbar mit `pip install paho-mqtt==2.1.0`
- Ubuntu-Pakete für Version 24.04:
 - Docker-Compose: falls `docker compose` noch nicht vorhanden ist (Check mit `docker compose -version`), dann `sudo apt install docker compose`
 - Mosquitto (MQTT-Broker), installierbar mit `sudo apt install mosquitto`
 - JDK, im Projekt JDK 17, Installationsanleitungen im Web
 - Maven (Java-Build-Tool), installierbar mit `sudo apt install maven`
- Visual Studio Code-Extensions zur Entwicklung:
 - Spring Boot Dashboard
 - Spring Boot Extension Pack
 - Spring Boot Tools
 - Spring Initializr Java Setup

Der Python-MQTT-Client Paho ist für den realen Betrieb nicht unbedingt auf dem PC erforderlich. Er stellt Python-Klassen bereit, die für die MQTT-Kommunikation verwendet werden können. Da sich die Python-MQTT-Komponenten auf dem STM befinden, wird dieses Paket nur für Debug-Zwecke benötigt.

Mosquitto ist der MQTT-Broker. Für einfache Zwecke kann mit dem obigen Befehl ein System-Daemon installiert werden. In Abschnitt 5.7 wird eine weitere Alternative mithilfe von Docker vorgestellt.

Das JDK (Java Development Kit) ist die Java-Entwicklungsumgebung. Es enthält unter anderem:

- den Java-Compiler (`javac`)
- die Java-Laufzeit (`java`)
- Standardbibliotheken
- Debugger und weitere Werkzeuge

Das JDK ist erforderlich für das Kompilieren und Ausführen des Spring-Boot-Projekts.

Maven ist ein Build- und Projektmanagement-Tool. Seine Aufgaben sind beispielsweise:

- Abhängigkeiten herunterladen
- Java-Dateien kompilieren
- Tests ausführen
- JAR-Dateien erzeugen
- Spring Boot starten (über ein Plugin)

Die `pom.xml` im `springboot`-Verzeichnis ist die Bauanleitung für Maven und enthält die notwendigen Pakete.

Für die Entwicklung wurde Visual Studio Code, weil dadurch die Nutzung der einfachen Projektverwaltung, einfachen Einrichtung und Integration der Spring-Boot-Tools, des Syntax-Hightlightings und der automatischen Codegenerierung gegeben ist.

Insbesondere die Tools für das Spring-Framework können in die `pom.xml` eingetragen und in VSC unter dem Reiter „Extensions“ installiert werden. Spring Boot ist ein Java-Framework. Es stellt u. a. bereit:

- REST-Controller
- Dependency Injection
- Konfigurationsverwaltung
- Embedded Tomcat
- JSON-Konvertierung
- Lifecycle-Management

Spring Boot bildet somit die Grundlage der Java-Anwendung.

Hier sind nur die PC-seitigen Abhängigkeiten aufgelistet. Die Abhängigkeiten für den STM wurden in Abschnitt 3.2 bereits erläutert.

5.3 Python-seitiger MQTT-Datenfluss

Die Python-Skripte aus dem Embedded-Systems-Teil werden um MQTT-Elemente erweitert. Während der Entwicklung wurde von unverschlüsseltem MQTT auf Secure-MQTT umgestiegen, um ein industriekonformes Projekt zu simulieren und mögliche Manipulationen zu minimieren. Im Folgenden werden die Begriffe „MQTT“ und „MQTT TLS“ synonym verwendet, obwohl sie technisch leicht unterschiedliche Varianten bezeichnen.

Als Grundlage für Python-MQTT-Elemente dienen die beiden Skripte `constants.py` und `utils.py`.

Ersteres stellt neben den für den reinen Steuerungsteil aus dem Embedded-Projekt benötigten Konstanten auch die Konstanten für die Verbindung mit dem Broker bereit, d. h. Port und IP des Brokers sowie den Pfad, unter dem die zur Authentifikation benötigten Zertifikate liegen.

Das `utils.py`-Skript stellt Routinen für die MQTT-Publisher und -Subscriber bereit, damit diese Funktionen nicht in jeder Komponente verteilt implementiert werden müssen. Dadurch werden Entwicklung und Wartung erleichtert. Folgende Funktionen wurden implementiert:

- **connect**: verbindet einen MQTT-Client mit dem MQTT-Broker und gibt den Client zurück. Für die Verbindung werden Zertifikate verwendet. Bei Bedarf kann auf die Authentifizierung mit Benutzername/Passwort gewechselt werden
- **example_publish**: es werden benutzerdefinierte Daten mit einer in diesem Skript einstellbaren Frequenz veröffentlicht.

- `publish`: eine benutzerdefinierte Nachricht wird veröffentlicht
- `subscribe`: startet einen blockierenden Subscriber
- `subscriber_in_thread`: ermöglicht nicht-blockierendes Abonnieren und bietet sich daher für einen nebenläufigen Betrieb an

Die Funktionen wurden genutzt, um die bestehenden Skripte aus dem `lerobot-` bzw. `lerobot-gui`-Ordner zu erweitern. Dafür wurden sie folgendermaßen integriert.

Die Python-Publisher in den Skripten `teleoperate.py` und `record_and_replay.py` umfassen die physikalischen Größen Position, Temperatur und Spannung des Leader- und des Follower-Arms. Diese Daten werden periodisch gesendet, d. h. in jedem Teleoperations-Loop. Für das Auslesen der Werte werden Methoden des LeRobot-Frameworks verwendet. Die Topics heißen wie folgt:

- Position: `leader/pos` und `follower/pos`
- Temperatur: `leader/temp` und `follower/temp`
- Spannung: `leader/volt` und `follower/volt`

Zudem gibt es auch Dummies für die Python-Publisher, damit das Debugging erleichtert und ein Betrieb ohne reale Roboterarme ermöglicht wird.

Des Weiteren existiert ein Subscriber für das Topic `dashboard/mode`. Dieser ist im Skript `dashboard_mode_control.py` implementiert. Dieses Skript dient zur Zentralverwaltung des Dashboard-Zustands. Das Dashboard wird in Abschnitt 6 beschrieben. Das Topic `dashboard/mode` wird verwendet, um mithilfe des Dashboards den Betriebsmodus zu bestimmen. Dadurch sollen Konflikte erkannt und Beobachtungs- bzw. Steuerungsmöglichkeiten mithilfe des Dashboards bereitgestellt werden.

Wichtig ist, dass die Arbeitsumgebung die in Abschnitt 5.2 aufgelisteten Anforderungen erfüllt. Ist dies der Fall, kann mit diesem Code in den Zielordner gewechselt werden:

```
cd <path_to_the_repo>
cd mqtt
```

Aus diesem Ordner können ein beispielhafter Dummy-Publisher sowie der passende Subscriber gestartet werden:

```
python3 leader_pos_publisher.py
python3 leader_pos_subscriber.py
```

Dies funktioniert aber nur, wenn der Broker-Server (siehe Abschnitt 5.7) bereits gestartet wurde. Des Weiteren ist der Code für Kommunikation von Client und Server in einem USB-Netzwerk mit der IP-Adresse `192.168.7.x` gedacht. Für das Debugging mit `localhost` müssen daher einige Anpassungen vorgenommen werden: Der Broker muss in `mqtt/constants.py` umgestellt werden. Des Weiteren müssen die Zertifikate in `mosquitto/mosquitto.conf` geändert werden.

5.4 Java-seitiger Datenfluss

Die Java-Dateien befinden sich im Package `com.mende.examination.soarms` innerhalb des `src/main/java`-Verzeichnisses des `springboot`-Ordners. Darin befinden sich einerseits Controller für die Interaktion mit der RESTful-API sowie MQTT-Subscriber und -Publisher. Weiterhin existieren Backend-Klassen zur Verarbeitung der physikalischen Größen sowie der Zertifikate. Die MQTT-Subscriber und -Publisher sind folgende:

- `MqttDashboardPublisher.java`
- `MqttRobotPublisher.java`
- `MqttRobotSubscriber.java`

Der `Dashboard-Publisher` veröffentlicht den aktuellen Dashboard-Zustand (Observe, Control oder Conflict) über MQTT auf dem Topic `dashboard/mode`. Die Steuerungsmethode bestimmt den Zustand des Dashboards. Der `MqttRobotPublisher` veröffentlicht die Gelenkkoordinaten der Slider im Control-Tab, wenn Control der aktuelle Dashboard-Modus ist. Der `MqttRobotSubscriber` abonniert die physikalischen Größen Position, Temperatur und Spannung der Robotergelenke auf den Topics `{id}/pos`, `{id}/temp` und `{id}/volt`.

Die beiden Java-Publisher sind anders aufgebaut als die Subscriber. Im Vergleich zur Vorlage aus [3] existieren in diesem Projekt keine `public void run(ApplicationArguments args)`-Methode und auch keine Implementierung des `ApplicationRunner`-Interfaces, damit erst die Verbindung vollständig abgeschlossen werden kann und erst dann veröffentlicht wird. Andernfalls ist der Verbindungsaufbau zu langsam und somit waren die ersten veröffentlichten MQTT-Messages ungültig, da noch keine Verbindung bestand.

Wenn der `MqttRobotSubscriber` Daten über Eclipse Paho empfängt, schreibt er sie in den `RobotStateService`. Dieser Service

- führt eine Positionshistorie (trägt gesampelte Werte ein, löscht die ältesten Werte, wenn Pufferlimit erreicht)
- nutzt die Klasse `RobotState` zur Verwaltung des aktuellen Zustands (der aktuellen physikalischen Größen)
- vererbt an `LeaderStateService` und `FollowerStateService`, um eindeutige Werte für die `deviceId`-Attribute setzen zu können

Da die Positionsdaten der beiden Roboterarme ausschlaggebend für die Regelung sind (Regel- und Führungsgröße) und Temperatur und Spannung nur alarmrelevant sind, wird ausschließlich die Historie der Positionen aufgezeichnet. Weiterhin weisen die Positionen eine deutlich schnellere Dynamik auf als Temperatur und Spannung. Zudem sind Spannung und Temperatur an den Motoren des Follower-Arms im Normalfall höher als am Leader-Arm, was einen Vergleich zusätzlich erschweren würde.

Die Leader- und Follower-Positionsdaten werden in `ArrayLists` gespeichert. Für das Anhängen neuer Listenelemente gilt $\mathcal{O}(1)$. Außerdem kann Elementzugriff über den Index mit $\mathcal{O}(1)$ erfolgen, was für das Plotten hilfreich ist. Lediglich beim Löschen des ältesten Elements wäre die `LinkedList` mit $\mathcal{O}(1)$ effizienter. Für den aktuellen Umfang ist die `ArrayList` aufgrund der einfachen Indexzugriffe ausreichend. Bei größeren Puffern oder höheren Datenraten wäre ein Ringpuffer auf Basis einer `ArrayDeque` oder eines eigenen

Ringpuffers effizienter, da das Entfernen des ältesten Elements dort keine linearen Verschiebungen verursacht. [6]

Um die Rechen- und Speicherlast unabhängig von der tatsächlichen Sensorabtastrate zu begrenzen, wird ein Sampling der eingehenden Messwerte durchgeführt. Dabei wird nur ein von `sampleRate` Messpunkten gespeichert. Um den RAM nicht zu sehr zu belasten, werden außerdem nur die letzten `bufferCapacity` Messpunkte in der Historie gespeichert. Die Variable `fps` bezeichnet die effektive Veröffentlichungsrate der Messwerte, die im Normal- und Optimalfall der Abtastrate der Sensorik entspricht. Mit den Parametern $B = \text{bufferCapacity}$, $S = \text{sampleRate}$ und $f_s = \text{fps}$ ergibt sich die Zeitdauer der Historie zu $T = \frac{B \cdot S}{f_s}$. Dabei entspricht `fps` der Abtastrate der Sensorik in $\frac{1}{s}$. Die Sample-Rate kann somit durch den Programmierer beliebig (in Grenzen) eingestellt werden. Wenn der Aufbau bspw. nicht nur zur Live-Daten-Visualisierung, sondern auch zum Training eines maschinellen Lernmodells verwendet werden soll, kann die Samplingrate auf 1 gesteigert werden, damit keine Daten verloren gehen.

5.5 REST-API und Spring Boot

REST (Representational State Transfer) beschreibt einen Architekturstil für verteilte Systeme. RESTful-APIs ermöglichen den Zugriff auf Ressourcen eines Servers durch einen Client. Die Kommunikation erfolgt typischerweise über das HTTP-Protokoll, wobei standardisierte HTTP-Methoden wie GET und POST verwendet werden. Ein wichtiges REST-Prinzip ist die Zustandslosigkeit (Statelessness), d. h. jede Anfrage enthält alle Informationen, die zur Verarbeitung benötigt werden und ist unabhängig von vorherigen Anfragen. [7]

Im Kontext dieses Projekts wird die REST-API unter anderem zum Aufruf der Zustandsdaten mittels HTTP verwendet. Diese liegen im JSON-Format vor und können über GET-Methoden abgefragt und bspw. im Browser angezeigt werden. Neben den Zustandsdaten des Systems werden auch der Dashboard-Modus oder die Slider-Werte für die Dashboard-seitige Programmierung des Followers (siehe Abschnitt 6.3) verwendet.

Somit wird der Austausch zwischen Java-Backend und den JavaScript-Skripten für das Frontend ermöglicht. Dafür werden Controller-Klassen mithilfe des Spring-Boot-Frameworks implementiert.

Es existieren folgende Controller:

- `DashboardController.java` implementiert GET, POST
- `WebControlController.java` implementiert GET, POST
- `HealthController.java` implementiert GET
- `RobotStateController.java` implementiert GET

Der `DashboardController` ermöglicht eine HTTP-POST-Methode auf `/api/dashboard/mode`. Wenn dort ein POST geschieht, werden die vom `MqttDashboardPublisher` empfangenen Daten über MQTT veröffentlicht. Weiterhin wird für den gleichen RESTful-API-Endpoint eine GET-Methode für den Erfolgsstatus der Anfrage auf dem gleichen Pfad bereitgestellt. Der `WebControlController` ermöglicht eine HTTP-POST-Methode auf `/api/control`, sodass bei einem POST die empfangenen Slider-Werte direkt vom

`MqttRobotPublisher` über MQTT veröffentlicht werden können. Die für denselben Endpoint bereitgestellte GET-Methode ermöglicht das Auslesen der Control-Daten. Zudem ermöglicht der `HealthController` eine HTTP-GET-Methode auf `/api/health` als Abfrage des Status des Servers. Der `RobotStateController` ermöglicht HTTP-GET-Methoden auf `/api/robot/{id}/state` (id ist die ID des Arms, nur „leader“ und „follower“ sind gültig), `/api/robot/{id}/posList` und `/api/robot/timeStampsList`. Mit diesen Mappings werden bei GET-Aufrufen über die RESTful-API die entsprechenden physikalischen Größen, die Positionshistorien und die Zeitstempel der Messwerte bereitgestellt, die zuvor durch die MQTT-Subscriber empfangen und im Backend gespeichert wurden.

Die Controller verwenden die folgenden in [8, 9] erklärten Annotationen:

- `@RestController`: kennzeichnet eine Klasse als REST-Controller. Die Rückgabewerte der Methoden werden automatisch als HTTP-Antwort (z. B. JSON) serialisiert
- `@RequestMapping`: definiert den Basispfad einer Controller-Klasse
- `@GetMapping`: ordnet eine Methode einer HTTP-GET-Anfrage zu
- `@PostMapping`: ordnet eine Methode einer HTTP-POST-Anfrage zu
- `@RequestBody`: wandelt den JSON-Body einer HTTP-Anfrage automatisch in ein Java-Objekt um

Da die Endpoints für die Verarbeitung mit JavaScript relevant sind, werden diese in Tabelle 5.1 zusammengefasst.

Endpoint	Beschreibung
<code>/api/dashboard/mode</code>	Dashboard-Modus (Observe/Control/Conflict)
<code>/api/robot/{id}/state</code>	physikalische Größen des aktuellen Zustands
<code>/api/robot/{id}/posList</code>	gesamte Positionshistorie (begrenzter Puffer)
<code>/api/health</code>	Aktivität des Servers
<code>/api/control</code>	aktuelle Slider-Werte in der GUI
<code>/api/robot/timeStampsList</code>	Zeitstempel der Messwerte

Tabelle 5.1: RESTful-API-Endpoints

5.6 Datenformat

Ursprünglich wurde ein Datenformat verwendet, bei dem der MQTT-Payload nur aus den Prozessdaten bestand, d. h. Zeichenketten als Schlüssel (Gelenkname mit der physikalischen Größe) sowie Werte im *double*- bzw. *float*-Format. Ein Beispiel dafür ist:

```
{
  "shoulder_pan": 5.32,
  "shoulder_lift": 27.56,
  "elbow_flex": 16.13,
  "wrist_flex": 23.36,
  "wrist_roll": 30.56,
  "gripper": 47.89
}
```

Um jedoch Zeitstempel für die Nachrichten zu verwenden, welche für das Plotten der Historie entlang der x-Achse erforderlich sind, sowie die Publisher über Device-IDs zu identifizieren, wurde das Datenformat erweitert. Somit besteht der Payload nun aus den Keys `processTimeStamp`, `deviceId` und `data`. Der `processTimeStamp` ist dabei eine Zeit in Sekunden als *float* bzw. *double*, die seit dem Systemstart vergangen ist. Die `deviceId` bezeichnet jedes physikalische Objekt, für das Daten veröffentlicht werden können und ist ein String. `data` ist wie oben im JSON-Format, wobei die Schlüssel jetzt zusätzlich die Endungen `.pos`, `.volt` und `.temp` tragen, um die zugehörige physikalische Größe darzustellen. Um das neue Format mit den Metadaten zu parsen, müssen die Daten eine Ebene tiefer geparkt werden. Daher wurde die Klasse `ParsedMessage` entworfen. Ein Beispiel für das neue Payload-Format ist:

```
{
  "processTimeStamp" : 10.52,
  "deviceId" : "leader",
  "data": {
    "shoulder_pan": 5.32,
    "shoulder_lift": 27.56,
    "elbow_flex": 16.13,
    "wrist_flex": 23.36,
    "wrist_roll": 30.56,
    "gripper": 47.89
  }
}
```

Für die Übertragung von Daten an REST-APIs wird häufig das JSON-Format verwendet, da es ein Industriestandard ist und dafür bereits viele Bibliotheken, bspw. für Python oder Java, existieren.

5.7 MQTT-Broker

Als MQTT-Broker wird Eclipse Mosquitto verwendet, weil dieser leichtgewichtig ist und auf verschiedenen Plattformen sowie ressourcenbeschränkten Geräten genutzt werden kann. Dadurch könnte der MQTT-Broker auch auf dem STM laufen. [10]

Um ein reproduzierbares Setup zu haben, das unabhängig von den systemweiten Installationen funktioniert, wurde ein Docker-Setup für Mosquitto eingerichtet. Es ist aber auch möglich, den System-Daemon aus Abschnitt 5.2 zu verwenden. Dieser funktioniert standardmäßig für Port 1883 und sollte bei den Befehlen unten an entsprechender Stelle beachtet werden.

Standardmäßig wird bei der Mosquitto-Installation über `apt` ein System-Daemon installiert. Dieser wird beim Systemstart automatisch im Hintergrund gestartet. Der Status des MQTT-Brokers kann mit

```
systemctl status mosquitto
```

überprüft werden. Falls der Daemon aktiv ist, kann er mit

```
sudo systemctl stop mosquitto
```

gestoppt werden. Zur Verwendung des Docker-Mosquitto-Containers muss Port 8883 für

MQTT über TLS (bzw. 1883 für MQTT) vom Host an den Container weitergeleitet werden. Diese Ports sind laut [11] die Standardports für MQTT über TLS bzw. MQTT. Miz

```
sudo systemctl disable mosquitto
```

startet Mosquitto nicht automatisch beim Boot-Vorgang. Anschließend wechselt man aus der Repo-Root mit

```
cd mosquitto
```

in den `mosquitto`-Ordner im Repo, da dieser die `docker-compose.yaml` enthält, die die Instruktionsanweisungen für den folgenden Befehl beinhaltet. Mit dem Befehl

```
docker compose up -d
```

kann nun die Docker-Anwendung gestartet werden. Die Wirkung kann anschließend überprüft werden, indem einer dieser 3 Befehle verwendet wird:

```
ss -tulpn | grep 8883      # 1883 ohne TLS
docker ps                 # zeigt Status der laufenden Container
docker inspect -f '{{.State.Status}}' mosquitto
```

Nach der Session kann der Container (und somit der Mosquitto-Server) mit

```
docker compose down
```

beendet werden. In der `docker-compose.yaml` wird dabei ein Service definiert, der das Image `eclipse-mosquitto:2` verwendet. Zudem findet ein Port-Mapping statt, damit der Broker die Ports nutzen kann. Weiterhin werden die Mosquitto-Konfigurationsdatei `mosquitto.conf` sowie die Zertifikate als Volumes in den Container eingebunden, damit der Mosquitto-Server im Docker-Container laufen kann. Die `mosquitto.conf`-Datei enthält die Konfiguration des Servers. Die Konfiguration definiert den TLS-Listener, die verwendeten Zertifikate sowie die Authentifizierungsmechanismen. Die Client-Zertifikatsauthentifizierung (Mutual TLS) wird aktiviert, indem `require_certificate true` gesetzt wird. Mit `use_identity_as_username true` wird der Common Name (CN) des Client-Zertifikats als MQTT-Benutzername verwendet. Anonyme Verbindungen werden durch `allow_anonymous false` verhindert, die Clients müssen sich also authentifizieren.

5.8 Zertifikate

Um eine stärkere Authentifizierungsmethode als Benutzername und Passwort zu verwenden, wurde die Authentifizierung mit Zertifikaten gewählt. Des Weiteren werden oft von einem Nutzer für unterschiedliche Dienste verschiedene Passwörter verwendet, was das Merken erschwert und Passwortzurücksetzungen erforderlich machen kann. Zudem können die Zertifikate lokal gespeichert und nach einmaligem Generieren oder Zustellen immer verwendet werden. [12]

Die Shell-Skripte zur Erstellung der Zertifikate und Schlüssel liegen im Verzeichnis `<repo_root>/certs`. Es gibt eine CA (Certification Authority), die Zertifikate signieren kann. Zudem besitzt jeder Client ein Zertifikat (`*.crt`) und einen Private Key (`*.key`) sowie die `ca.crt`-Datei. Für jeden Server existiert eine `*.crt`- und eine `*.key`-Datei, wobei das Zertifikat mithilfe der Konfigurationsdateien `san.cnf` erstellt wird.

Für die Generierung der Private- und Public-Keys sowie der CAs wurde OpenSSL als CLI-Tool verwendet. Um die Dateien reproduzierbar selbstständig generieren zu können, wurden Shell-Skripte geschrieben. Im Verzeichnis `<repo_root>/certs` müssen dafür die folgenden Befehle ausgeführt werden:

```
chmod +x create_all_certs.sh
./create_all_certs.sh
chmod +x distribute_all_certs.sh
./distribute_all_certs.sh
```

Damit werden weitere Shell-Skripte ausgeführt, die zuerst die CA und danach die Zertifikate erstellen. Zusätzlich werden die Java-Client-Zertifikate in Keystore-Zertifikate konvertiert. Diese enthalten den Key und das Zertifikat. Zudem benötigt man für Java noch einen Truststore, der alle vertrauenswürdigen CAs enthält. Keystores enthalten private Schlüssel und Zertifikate zur eigenen Identifikation. Truststores enthalten Zertifikate von vertrauenswürdigen Zertifizierungsstellen und dienen zur Überprüfung anderer Zertifikate. [13], [14]

Abschließend werden die Zertifikate durch die Skripte automatisch in die passenden Projektordner verteilt. Da die Zertifikate für den Docker-Container und den STM auf dem Host erstellt wurden, hat der Mosquitto-Docker keinen Zugriff, da er vom User `mosquitto` betrieben wird. Daher muss erst in der Ordner `<repo_root>/mosquitto/certs` gewechselt werden. Die Rechte können folgendermaßen freigegeben werden:

```
chmod 644 *.crt
chmod 640 *.key
```

Somit verfügt der Docker-Container über die notwendigen Zugriffsrechte. Dies ist jedoch nicht industrienaah, da nicht alle Benutzer Lesezugriffe für die Keys haben sollen. Alternativ könnte man den Benutzer `mosquitto` zu einer Gruppe hinzufügen, die die nötigen Leserechte für die Zertifikate hat. Auf dem STM besteht dasselbe Problem, dieses wurde jedoch schon im Bitbake-Layer in Abschnitt 3 behoben.

Schlussendlich liegen alle Client- und Server-Zertifikate und die Dateien `ca.key` und `ca.crt` im Verzeichnis `<repo_root>/certs`. Das CA-Zertifikat und alle Python-Client-Zertifikate sowie -Schlüssel werden ins Verzeichnis `<repo_root>/meta-lerobot/recipes-lerobot/lerobot/files/certs` kopiert. Außerdem wird das CA-Zertifikat zusätzlich mit den Zertifikaten und Schlüsseln für den Server sowie für die Mosquitto-Clients ins Verzeichnis `<repo_root>/mosquitto` kopiert.

6 Web-Dashboard

6.1 Überblick

Das Web-Dashboard verwendet die folgenden Technologien:

- HTML
- CSS
- JavaScript
- REST-API-Integration
- Live-Updates alle 400 ms
- automatische Aktualisierung offener Tabs nach Neustart der Anwendung

Das Dashboard ist über die URL `localhost:8080/index.html` erreichbar. Das Spring-Boot-Backend startet beim Programmstart automatisch einen eingebetteten Tomcat-Webserver auf Port 8080. Dieser stellt sowohl die RESTful-API als auch die statischen HTML-, CSS- und JavaScript-Dateien bereit. Das Dashboard verfügt über drei Reiter:

- General: `index.html`
- Diagram: `diagram.html`
- Control: `control.html`

Das Frontend wird mit HTML, CSS und JavaScript umgesetzt, wobei ausschließlich Daten aus der RESTful-API verwendet werden. JS stellt zusammen mit der REST-API den Übergang zwischen dem Backend und dem Frontend dar. Die HTML-Dateien liegen im Ordner `<repo_root>/src/main/resources/static`, die CSS-Dateien im Unterverzeichnis `css` und die JS-Dateien im `js`-Verzeichnis.

Nachfolgend werden die Seiten selbst sowie die daraus resultierenden Steuerungsmöglichkeiten vorgestellt.

- *General*: Für den Leader- und den Follower-Arm werden Tabellen mit den aktuellen Größen Position der Motorwelle sowie die Temperatur und die Spannung an den sechs Motoren dargestellt. Dies zeigt Abbildung 6.1.
- *Diagram*: Es wird der zeitliche Verlauf der Position eines auswählbaren Motors der beiden Arme visualisiert. Eine Beispieldarstellung dafür ist in Abbildung 6.2 dargestellt.
- *Control*: Mithilfe der 6 Slider kann der reale Follower-Arm gesteuert werden, wobei man die Motorwellen nur einzeln bewegen kann. Dieser Reiter ist in Abbildung 6.3 zu sehen.

Die Update-Frequenz der Seiten *General* und *Diagram* betragen standardmäßig 400 ms und sind über die `setInterval`-Funktionen in den JS-Skripten einstellbar. Für jedes Fenster gibt es eine HTML-Datei mit einer zugehörigen JavaScript-Datei.

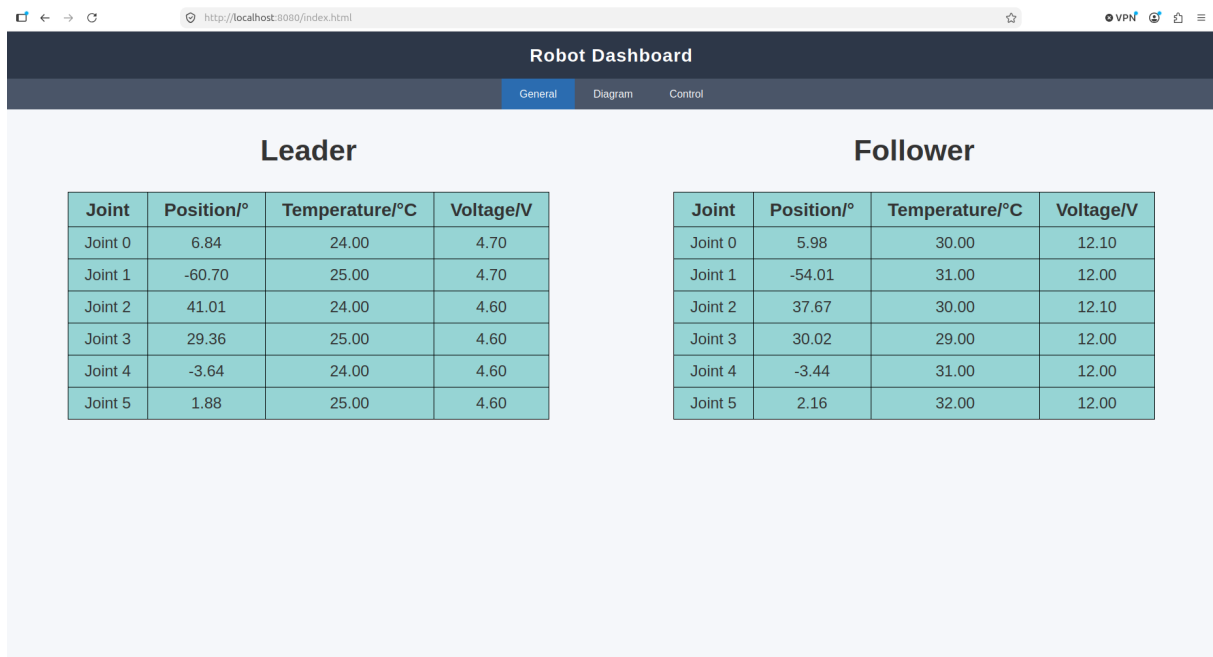


Abbildung 6.1: Beispielbild für den Reiter General auf der Website



Abbildung 6.2: Beispielbild für den Reiter Diagram auf der Website

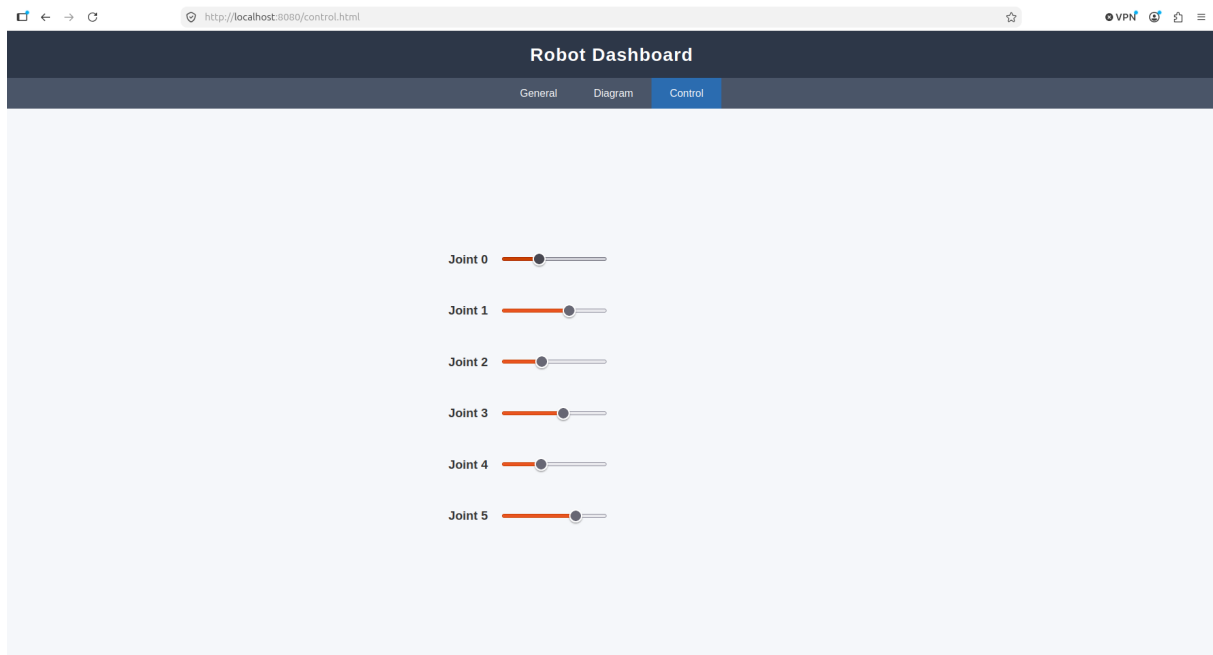


Abbildung 6.3: Beispielbild für den Reiter **Control** auf der Website

6.2 Auto-Refresh der offenen Tabs

Der Auto-Refresh der offenen Tabs nach Serverneustart ist aus zwei Gründen wichtig:

1. Der Frontend-Zustand wird komplett zurückgesetzt. Dies ist unter anderem für die Anordnung der Alarmmeldungen essenziell. Außerdem werden keine veralteten Daten mehr angezeigt.
2. Über den Health-REST-Endpoint `/api/health` ist ersichtlich, ob mit dem Server kommuniziert werden kann.

Sowohl die Seite `index.html` als auch `diagram.html` verfügen über den Auto-Refresh, der jeweils in den zugehörigen JS-Skripten implementiert ist.

Der Check, ob der Server neu gestartet wurde, ist in der JS-Funktion `checkServerRestart` implementiert. Am wichtigsten ist hierbei der `fetch` des Health-Endpoint. Wenn der REST-Endpoint nicht gefetcht werden kann, ist der Tomcat-Server inaktiv. Falls der Fetch erfolgreich ist, der Server inaktiv war und noch kein Reload durchgeführt wurde, liegt ein Serverneustart vor. In diesem Fall wird die Seite mit `location.reload()` neu gestartet.

6.3 Betriebsmodi

Die geöffneten Dashboard-Fenster im Browser ergeben den Dashboard-Modus. Dieser bestimmt, wie der Follower-Arm gesteuert wird. Es gibt die folgenden 3 Möglichkeiten:

- Wenn nur Observationsfenster, d. h. **General**- oder **Diagram**-Fenster, geöffnet sind, befindet sich das Dashboard im Observations-Modus, sodass die GUI auf dem STM die Kontrolle über die Steuerung hat.
- Wenn nur ein **Control**-Fenster geöffnet ist, befindet sich das Dashboard im Control-Modus. In diesem Fall kann der Follower mithilfe der Slider gesteuert werden.

- Wenn mehrere **Control**-Fenster geöffnet sind, ist nicht bekannt, welche Slider verwendet werden sollen, daher löst dies einen Konflikt aus. Ein Konflikt besteht ebenfalls, wenn Observations- und Kontrollfenster gleichzeitig offen sind, da dann nicht klar ist, ob der Follower über die GUI auf dem STM oder über die Slider auf der Website gesteuert werden soll. Im Konflikt-Modus gibt es keine aktive Steuerungsmethode, weshalb eine Fehlermeldung auf dem STM-Display ausgegeben wird.

Zusammengefasst gelten folgende Regeln:

- observe-Modus: Es ist kein Browser-Fenster oder es sind nur Beobachtungsfenster geöffnet.
- control-Modus: Es ist genau ein control-Fenster geöffnet.
- conflict-Modus: Es sind ein control-Fenster und mindestens ein beliebiges weiteres Fenster geöffnet.

Die Erkennung des aktiven Modus findet mithilfe der JavaScript-Klasse **TabManager** statt, die in den JavaScript-Dateien für die Fenster jeweils einmalig instanziiert wird. Jeder Tab erhält eine 128-Bit lange UUID. Der **TabManager** zählt die geöffneten Tabs pro Seitentyp (**index/diagram/control**) und erstellt ein Mapping zwischen Tab-UUID und Seitentyp. Daraus wird nach den oben genannten Regeln der Dashboard-Modus bestimmt. Dieser kann mit der Methode **getMode()** ausgegeben werden. Die Zählung findet statt, indem beim Aufruf jedes Tabs die **start**-Methode der **TabManager**-Objekte aufgerufen wird. Dabei wird eine „hello“-Nachricht auf dem Channel gepostet. Dadurch wird jeder aktive **TabManager** der Seite getriggert. In den einzelnen Listen wird der neue Tab registriert und die aktiven Tabs antworten mit einer „I am“-Nachricht. Außerdem werden die **lastSeen** Werte der Tabs-Elemente geändert. Beim Schließen von Tabs erfolgt das Versenden einer „closed“-Nachricht, die die Tabs aus der Liste löscht. Zudem gibt es einen Heartbeat, der jede Sekunde eine „alive“-Nachricht versendet. Dadurch können abgestürzte Tabs, bei denen keine **close**-Methode aufgerufen wurde, oder solche, bei denen die **start**-Methode nicht geklappt hat, eingetragen werden. Wenn ein Tab 3 Sekunden lang keinen Heartbeat beantwortet, wird sie aus der Liste der aktiven Tabs gelöscht.

Über das Skript `/opt/lerobot-gui/src/dashboard_mode_control.py` erfolgt die Zugriffskontrolle auf dem STM. Dieses interpretiert für die jeweiligen Betriebsmodi entsprechende Skripte in separaten Prozessen. Außerdem werden bereits existierende Prozesse mit **SIGINT** bzw. **SIGTERM** beendet. Wenn sich der Dashboard-Modus ändert, wird er über die **getMode**-Methode der **TabManager**-Klasse aus JavaScript in der REST-API auf dem Endpoint `/api/dashboard/mode` gepostet. Anschließend wird er von einem Java-MQTT-Publisher auf `dashboard/mode` veröffentlicht. Ein Python-MQTT-Subscriber auf dem STM abonniert `dashboard/mode`. Dieser Python-Subscriber läuft in einem Thread, um das Programm nicht am Ausführen der eigentlichen Funktion zu hindern. Beim Empfangen einer Nachricht auf `dashboard/mode` wird ein Callback ausgeführt, das ein globales Objekt im `dashboard_mode_control.py`-Skript ändert. Bei einer Änderung wird der laufende Prozess beendet, sodass es keine Konflikte konkurrierender Prozesse gibt. Außerdem wird das entsprechende Python-Skript interpretiert und somit die GUI umgeschaltet. Im **control**-Modus werden die Roboterarme über die Slider im **Control**-Fenster der Website gesteuert. Die eigentliche Interaktion zur Behebung des **conflict**-Modus bzw. zur Steuerung im **control**-Modus erfolgt aber über den Browser. Auf dem STM werden die Fenster aus den Abbildungen 4.9 und 4.10 angezeigt. In den anderen beiden Tabs (**General** und **Diagram**) erfolgt die Steuerung durch die GUI auf dem STM.

6.4 Alarm

Da die Temperaturen und Spannungen der Motoren ausschlaggebende Indikatoren für den Zustand der Motoren und der Spannungsversorgung sind, wurden sowohl für die Temperatur als auch für die Spannung Schwellwerte eingeführt. Bei außerhalb des definierten Bereichs liegenden Messwerten wird davon ausgegangen, dass entweder die Sensorik oder die Spannungsversorgung fehlerhaft ist. Für die Temperatur und Spannung gibt es jeweils für den Follower- und den Leader-Arm einen oberen und einen unteren Schwellwert. Für die Definition sinnvoller Schwellwerte wurde ein Testbetrieb über 30 min durchgeführt. Diese Schwellwerte sind in der `alarm.js`-Datei im `js`-Verzeichnis des `static`-Ordners gespeichert. Es gelten die in Tabelle 6.1 spezifizierten Grenzwerte. Erst eine Unter- bzw. Überschreitung dieser Schwellwerte führt zu einer entsprechenden Alarmmeldung.

Tabelle 6.1: Schwellwerte für Alarmmeldungen

Kriterium	Temperatur / °C		Spannung / V
	Warnung	Kritisch	Kritisch
Untergrenze Leader	-	5	4,5
Obergrenze Leader	50	55	5,5
Untergrenze Follower	-	5	11,5
Obergrenze Follower	55	60	12,5

Die Anzeige des Alarms ist für das **General**- und das **Diagram**-Fenster implementiert, da nur diese beiden Fenster zur Beobachtung entworfen wurden.

Der Alarm ist innerhalb des JavaScript-Layers implementiert, da somit keine zusätzlichen Daten veröffentlicht oder abonniert werden müssen. Somit musste nur die Verarbeitungslogik auf Basis der mit dem `fetch`-Befehl aus der RESTful-API gewonnenen Daten entworfen werden.

Die aktuellste Alarmmeldung wird immer ganz oben angezeigt. Von einem Typen eines Alarms (also bspw. Temperatur auf dem Leader-Arm) kann außerdem immer nur eine Meldung (kritisch oder Warnung) gleichzeitig angezeigt werden. Kritische Meldungen erscheinen mit rotem, Warnungen mit gelbem Text. Für die Sortierung der Alarmer wurden IDs für die Alarmer eingeführt, die nach Eintrittszeit vergeben werden, da die Alarmer somit zeitlich geordnet werden. Es können alle vier Alarmer angezeigt werden, wobei verstrichene Alarmer nicht mehr angezeigt werden, sondern nur aktuelle. Ein Beispielbild für die Anzeige aktiver Alarmer ist auf Abbildung 6.4 dargestellt. Hierfür wurden die Schwellwerte kurzzeitig überschrieben, um die Warnung visualisieren zu können.

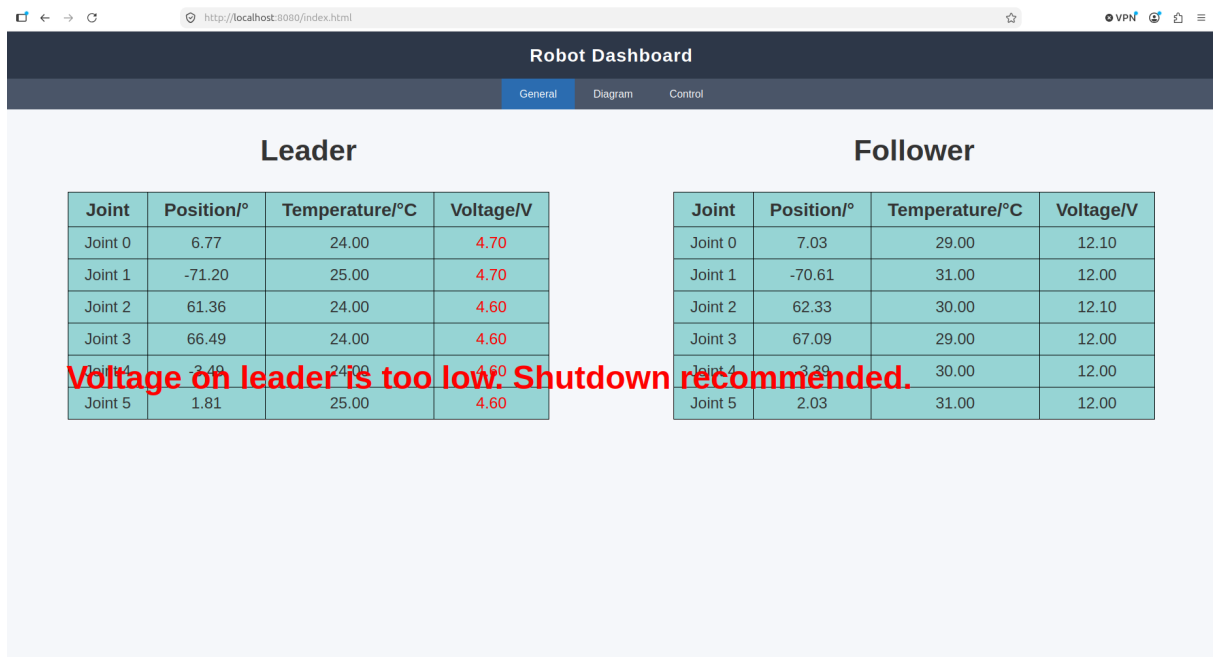


Abbildung 6.4: Beispielhafte Alarmmeldung im Reiter General

7 Inbetriebnahme

In diesem Kapitel wird die Inbetriebnahme des gesamten Systems beschrieben. Als Ausgangspunkt dafür dienen die Beschreibungen in Abschnitt 7.1. Falls die Bedienung über das Dashboard auf der Website gewünscht ist, muss zusätzlich das Setup aus Abschnitt 7.2 durchlaufen werden. Abschließend kann die Inbetriebnahme des Gesamtsystems anhand der Beschreibungen aus Abschnitt 7.3 erfolgen.

7.1 Installation

Zuerst wird das Repository geklont. Dafür wird dieser Befehl genutzt:

```
git clone https://github.com/LennartMende/so101-standalone-ctrl
```

Anschließend wechselt man in den Ordner `<repo_root>/certs` und befolgt die Anleitung aus Abschnitt 5.8, die hier zusammengefasst wird. Mit den folgenden Befehlen generiert man die Zertifikate und verteilt sie in die passenden Verzeichnisse:

```
chmod +x create_all_certs.sh
./create_all_certs.sh
chmod +x distribute_all_certs.sh
./distribute_all_certs.sh
```

Zusätzlich muss dafür gesorgt werden, dass die Nutzer über die Zugriffsrechte für die Dateien verfügen. Dafür werden im Verzeichnis `<repo_root>/mosquitto/certs` die folgenden Befehle ausgeführt:

```
chmod 644 *.crt
chmod 640 *.key
```

7.2 Einmaliges Dashboard-Setup

Um einmalig die MQTT-Kommunikation und die Einbindung des Dashboards zu starten, müssen die folgenden Schritte durchgeführt werden:

1. Falls Mosquitto in einem Docker-Container ausgeführt werden soll, wechselt man erst aus `<repo_root>` in `mosquitto`. Dort startet man mit `docker compose up -d` den Mosquitto-Broker. Andernfalls kann man den System-Daemon starten, falls er noch nicht gestartet ist.
2. Man überprüft mit

```
ss -tulpn | grep 8883
```

ob der MQTT-Broker für MQTT über TLS auf Port 8883 läuft. Für MQTT ohne Verschlüsselung muss man auf Port 1883 prüfen. Wenn der Broker aktiv ist, erscheint als Feedback ein Listener auf diesem Port.

3. Nachdem der STM hochgefahren ist (bei einer USB-Verbindung muss der Host über eine Schnittstelle im Netzwerk 192.168.7.x verfügen) wechselt man im Terminal in den `<repo_root>/springboot`-Ordner und gibt den Befehl

```
mvn spring-boot:run
```

ein. Dadurch laufen die PC-seitigen Java-Subscriber sowie der eingebettete Tomcat-Server auf <https://localhost:8080>, der für die Visualisierung und die REST-API erforderlich ist.

4. Man öffnet einen Browser. Beispielhafte REST-Endpoints sind unter den URLs localhost:8080/api/robot/state und localhost:8080/api/robot/posList erreichbar. Außerdem sind die von den Publishern veröffentlichten JSON-Daten einsehbar. Darüber hinaus kann mit localhost:8080/index.html die Webseite des Dashboards im Reiter **General** angezeigt werden. Es dürften noch keine Positionen, Temperaturen und Spannungen angezeigt werden. Deshalb werden diese Zellen in der Tabelle mit einem „-“ gekennzeichnet.

Anschließend kann eine Sensordatensimulation durchgeführt werden. Dazu muss der Broker auf **localhost** umgestellt werden. Man öffnet ein weiteres Terminal, um in das **mqtt**-Verzeichnis des Repos navigieren zu können. Wenn nötig, muss der Bediener in die entsprechende Python-Umgebung mit dem Python-Paho-Client wechseln. Anschließend wird der Python-Publisher mit dem Befehl

```
python3 leader_pos_publisher.py
```

gestartet. Dieser veröffentlicht simulierte Daten auf dem Topic „**leader/position**“. Die simulierten Daten werden im Dashboard live aktualisiert. Außerdem erfolgt die Aktualisierung der Messwerte beim Klicken der Refresh-Buttons im Browser auf der REST-API-Seite localhost:8080/api/robot/posList.

7.3 Betrieb des gesamten Systems

Unter <https://nextcloud.eit.htwk-leipzig.de/index.php/s/BfE2Ga9As6mmBBd> ist ein Demonstrationsvideo abgelegt, welches das Vorgehen in diesem Abschnitt anschaulich zeigt. Außerdem liegt dort ein SD-Karten-Image ab, welches mithilfe des letzten Schrittes aus Abschnitt 3.3 auf die SD-Karte geflasht werden kann, falls dies noch nicht erfolgt ist.

Die SD-Karte mit dem geflashten Image wird in den STM eingelegt. Wenn der STM mit Spannung versorgt ist, wird automatisch der Startvorgang eingeleitet. Nach einer kurzen Wartezeit erscheint das Setup-Fenster der GUI. Hier wird der Anwender durch den Einrichtungsprozess geführt. Es müssen beide Roboterarme per USB verbunden und mit Spannung versorgt sein. Außerdem wird der Follower-Arm in die Ruheposition gesetzt. Falls im Browser kein Fenster oder nur Beobachtungsfenster (General- bzw. Diagramm-Fenster) geöffnet sind, liegt die Kontrolle über die Roboterarme beim STM. Auf der GUI kann dann zwischen den Steuerungsmethoden **Teleop** und **Record & Replay** gewechselt werden. Dadurch werden MQTT-Publisher gestartet, die automatisch Prozessdaten über MQTT veröffentlichen. Diese Daten können über die REST-Endpoints aus Tabelle 5.1 oder über das Dashboard eingesehen werden. Das Dashboard aktualisiert die Daten live. Beim erneuten Abrufen der REST-Endpoints über GET-Methoden (entweder mit **curl** oder im Browser) werden die aktuellen Daten angezeigt. Außerdem kann der Follower über die Slider im **Control**-Fenster der Website gesteuert werden.

8 Zusammenfassung und Ausblick

8.1 Zusammenfassung

In dieser Belegarbeit wurde das Robotersystem SO-ARM101 um eine lokale Steuerungsoberfläche sowie eine verteilte Architektur zur Übertragung, Verarbeitung und Visualisierung von Betriebsdaten erweitert. Für den STM32MP135F-DK wurde mittels Yocto ein vollständiges Linux-System mit allen benötigten Softwarepaketen erstellt. Durch ein eigenes BitBake-Layer mit den notwendigen Recipes konnten sowohl das LeRobot-Framework als auch die grafische Benutzeroberfläche integriert werden. Die GUI führt den Anwender durch den Systemeinrichtungsprozess. Außerdem ermöglicht die GUI eine teleoperative Steuerung des Roboters sowie die Aufnahme und Wiedergabe von Trajektorien.

Die dabei erfassten Sensordaten der Motoren werden über MQTT übertragen, in einem Java-Spring-Boot-Backend verarbeitet und über eine REST-API bereitgestellt. Eine Webseite visualisiert diese Daten live und ermöglicht die Steuerung des Follower-Arms über Slider. Außerdem wird die Steuerung über die GUI blockiert, wenn sich das Dashboard im **Control**- und **Conflict**-Modus befindet. Ergänzend wurden Zertifikate für eine TLS-gesicherte MQTT-Kommunikation generiert und in die Systemumgebung integriert.

8.2 Ausblick

Das in diesem Projekt entwickelte System mit den Roboterarmen wurde bereits in einer Schule verwendet. Darüber hinaus soll es zukünftig auf Messen demonstriert werden. Aktuell steht der HTWK kein Rechner zur Verfügung, mit dem die Visualisierung der Daten über die Website ermöglicht wird. Dementsprechend sollte ein Laptop eingerichtet werden, um auch dieses Ergebnis des Projekts verwenden zu können. Darüber hinaus wurde sich ein Demonstrationsmodus gewünscht, bei dem der Follower eine vorgegebene Trajektorie abspielt, beispielsweise zum Winken.

Darüber hinaus ist die Bedienung des Follower-Arms über die Slider im **Control**-Fenster zu überarbeiten. Aktuell startet der Follower in der Home-Position jedes Gelenks. Allerdings sollte der Follower wie zu Beginn der Teleoperation in der Ruhepose starten. Darüber hinaus sollte ein Filter implementiert werden, der die ruckartigen Bewegungen des Followers verhindert. Die Ansteuerung der Ruhepose ist außerdem kritisch, da es zu ruckartigen Bewegungen des Followers kommt. Zudem kann die gleichzeitige Regelung der Gelenke zu Kollisionen mit dem Brett oder sonstigen Objekten im Arbeitsraum führen. Aus diesem Grund sollte eine stabilere und sanftere Ansteuerung der Ruhepose erfolgen, beispielsweise über die sequenzielle Ansteuerung von Zwischenpositionen.

Weitere kleine Punkte für die weitere Bearbeitung sind das Anzeigen des Systemzustands auf der Website sowie das Konvertieren der Fenster aus den Abbildungen 4.9 und 4.10 in das allgemeine Layout. Dadurch würde das HTWK-Logo oben links zu sehen sein und außerdem könnte die Sprache oben rechts umgeschaltet werden.

Literaturverzeichnis

- [1] URL: <https://core-electronics.com.au/attachments/uploads/sts3215-smart-servo-datasheet-translated.pdf> (besucht am 29.07.2026).
- [2] URL: <http://doc.feetech.cn/#/proinfo/download?srcType=FT-SMS-STS-emanual-229f4476422d4059abfb1cb0> (besucht am 29.07.2026).
- [3] Prof. Dr.-Ing. Andreas Pretschner. *Embedded Systems III und Verteilte Systeme*. Vorlesungsunterlagen. Nicht veröffentlichte Materialien. 2026.
- [4] URL: <https://github.com/STMicroelectronics/oe-manifest> (besucht am 10.08.2026).
- [5] URL: <https://github.com/STMicroelectronics/meta-st-pkgrepo-updates/blob/openstlinux-ecosystem-5.0.0/recipes-python/pyqt5> (besucht am 14.08.2026).
- [6] URL: <https://www.baeldung.com/java-arraylist-linkedlist> (besucht am 22.07.2026).
- [7] URL: <https://www.ibm.com/de-de/think/topics/rest-apis> (besucht am 26.07.2026).
- [8] URL: <https://www.geeksforgeeks.org/advance-java/spring-rest-controller/> (besucht am 27.07.2026).
- [9] URL: <https://www.geeksforgeeks.org/advance-java/spring-postmapping-and-getmapping-annotation/> (besucht am 26.07.2026).
- [10] URL: <https://mosquitto.org/> (besucht am 25.07.2026).
- [11] URL: <https://github.com/eclipse-mosquitto/mosquitto/blob/master/mosquitto.conf> (besucht am 28.07.2026).
- [12] URL: <https://nvlpubs.nist.gov/nistpubs/specialpublications/nist.sp.800-52r2.pdf> (besucht am 28.07.2026).
- [13] URL: <https://help.internetx.com/display/SSL/OpenSSL+-+Die+wichtigsten+Befehle> (besucht am 28.07.2026).
- [14] URL: <https://www.baeldung.com/java-keystore-truststore-difference> (besucht am 28.07.2026).

Für das Schreiben der Service-Dateien in Abschnitt 3.2.3 wurde ChatGPT von OpenAI als Hilfestellung verwendet.